CS 322: COMPUTER VISION

Encoders, Decoders, Unsupervised Self-supervised



COMP 322: Computer Vision and Image Processing

Hong Xu · Rhodes College

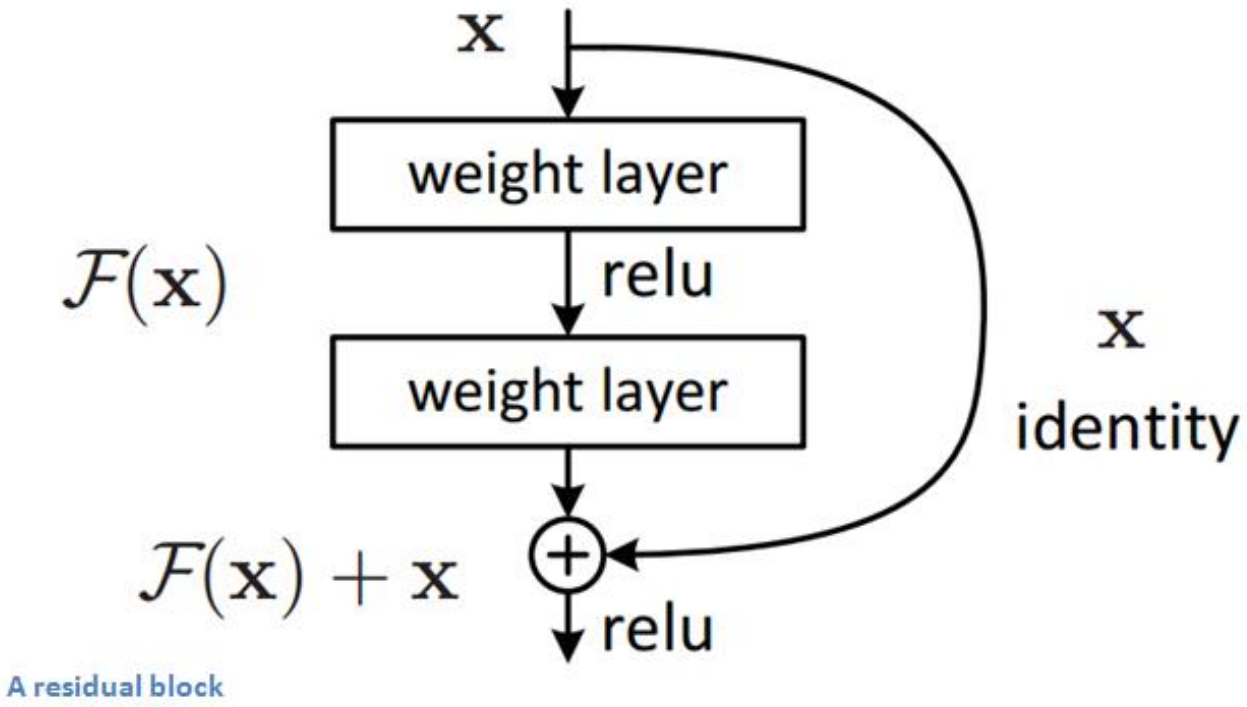
Adapted from Brown University CSCI 1430 Computer Vision (Spring 2026).

Original file: 2026_Spring_L18_EncodersDecoders_UnSelfSupervised.pptx.

Image Credit: Pixar Animation Studios

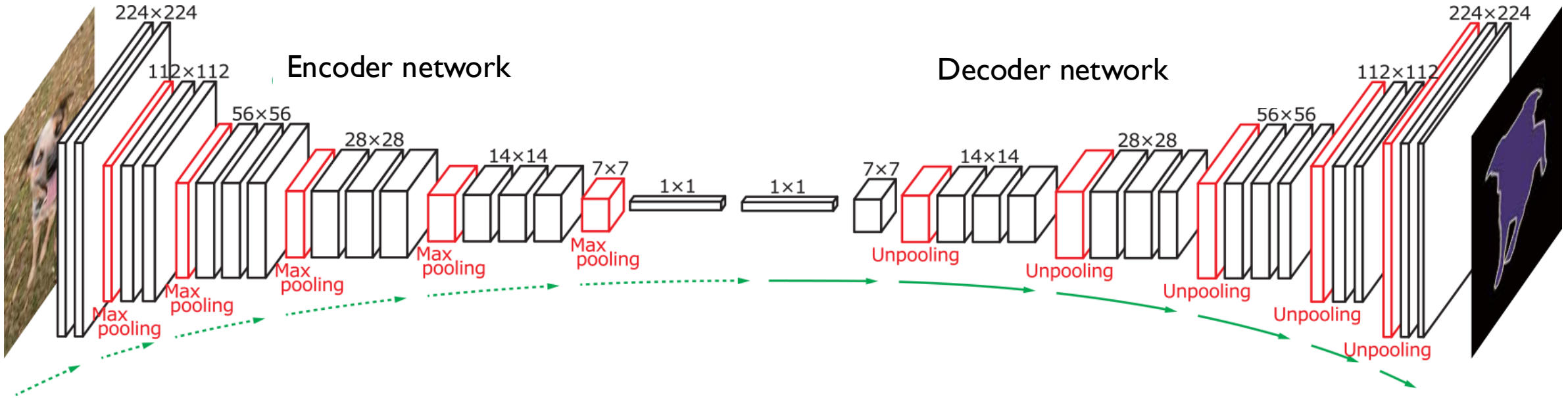
Residual Unit

The inputs of a lower layer is added to a node in a higher layer.



A residual block

Decoder networks *learn to upsample*



Often called “deconvolution”, but misnomer.
Not the deconvolution that we saw in deblurring -> that is division in the Fourier domain.

‘Transposed convolution’ is better.

Zeiler et al., Deconvolutional Networks, CVPR 2010
Noh et al., Learning Deconvolution Network for Semantic Segmentation, ICCV 2015

What is ‘in’ an encoder?

What is model inspection?

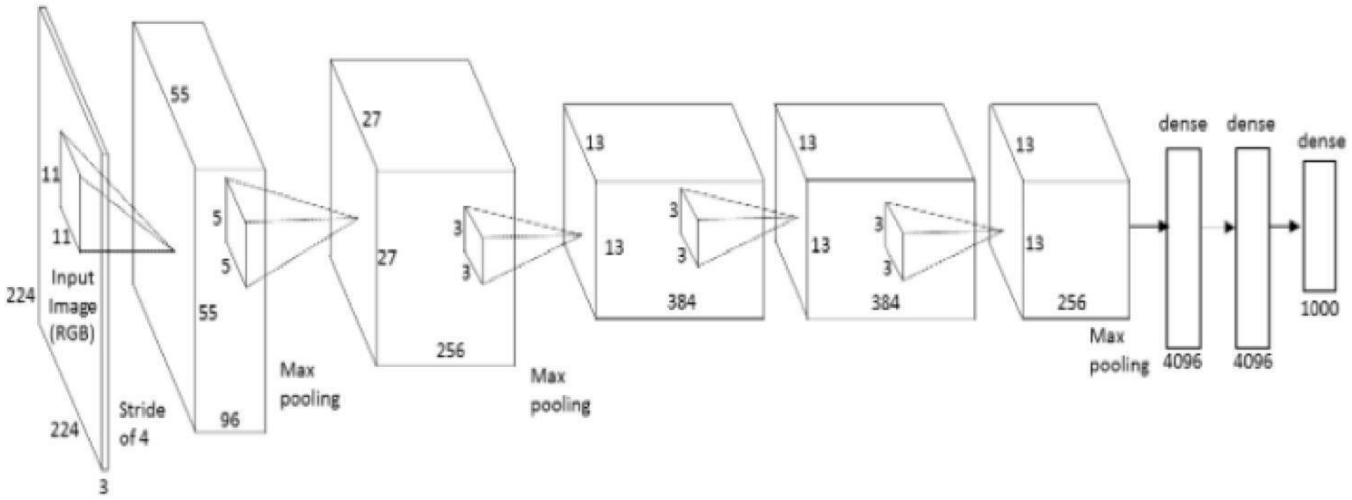
I want to know *how* my model is performing.

What could I do?

- Compute statistics, precision/recall plots
- Look at confusion matrices / misclassified images

But which neurons are important?
What *within* the images themselves is important?

What input to a neuron maximizes a class score?

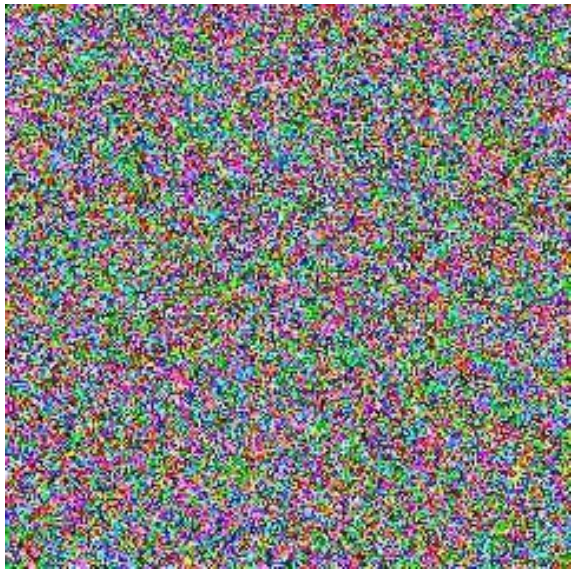


To visualize the function of a specific unit in a neural network, we synthesize an input to that unit which causes high activation.

Neuron of choice *i*
An image of random noise *x*.

Repeat:

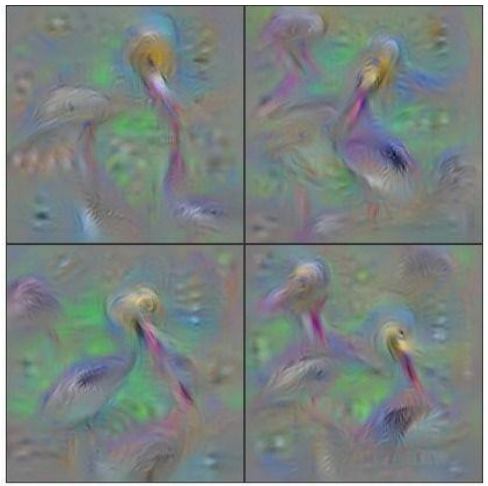
1. Forward propagate: compute activation $a_i(x)$
2. Back propagate: compute gradient at neuron $\partial a_i(x) / \partial x$
3. Add small amount of gradient back to noisy image.



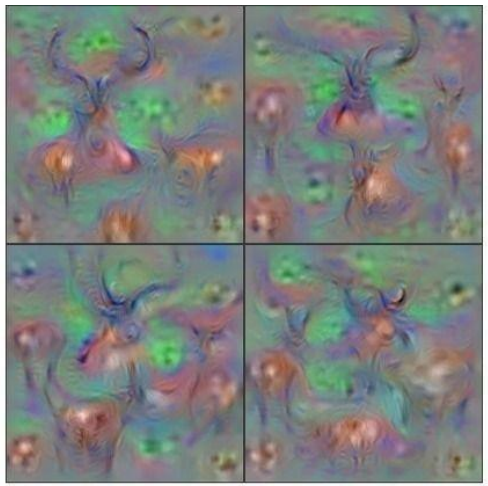
What image maximizes a class score?



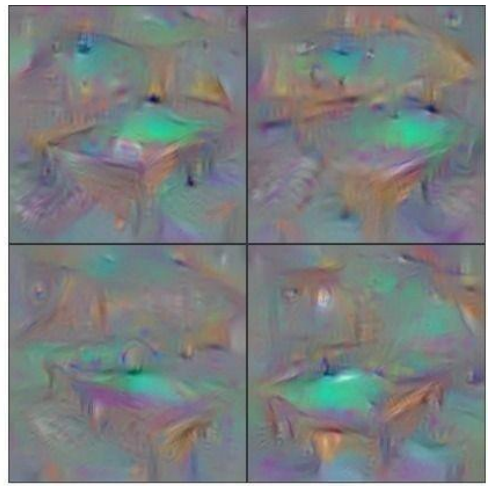
Flamingo



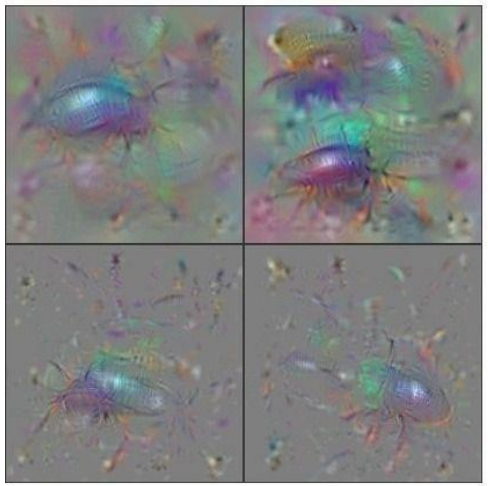
Pelican



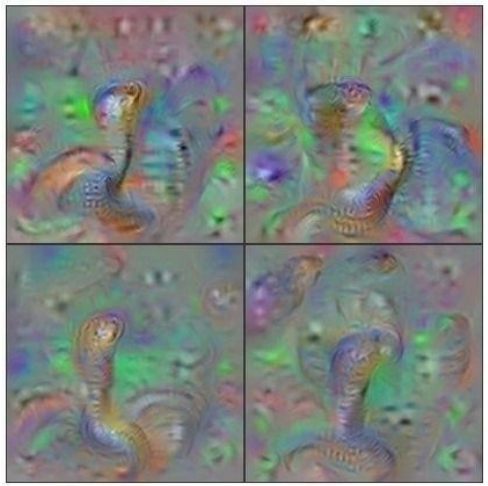
Hartebeest



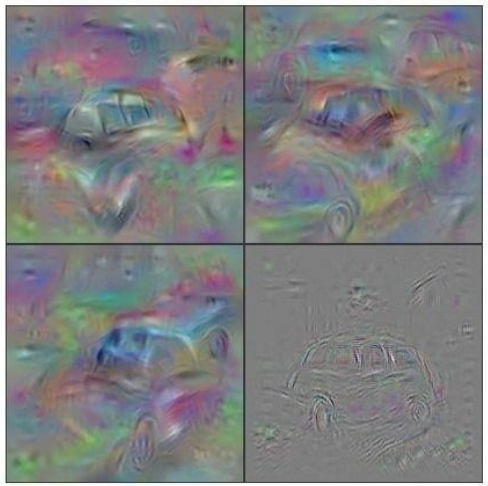
Billiard Table



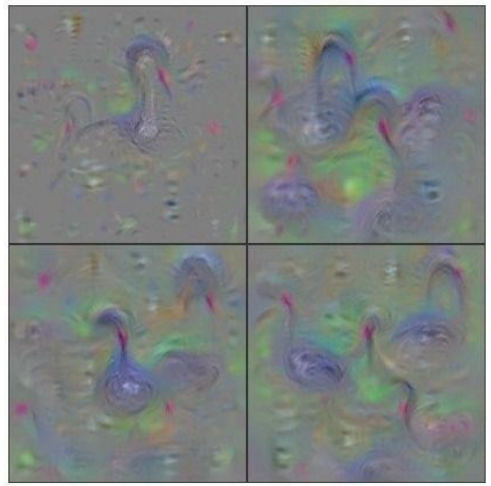
Ground Beetle



Indian Cobra



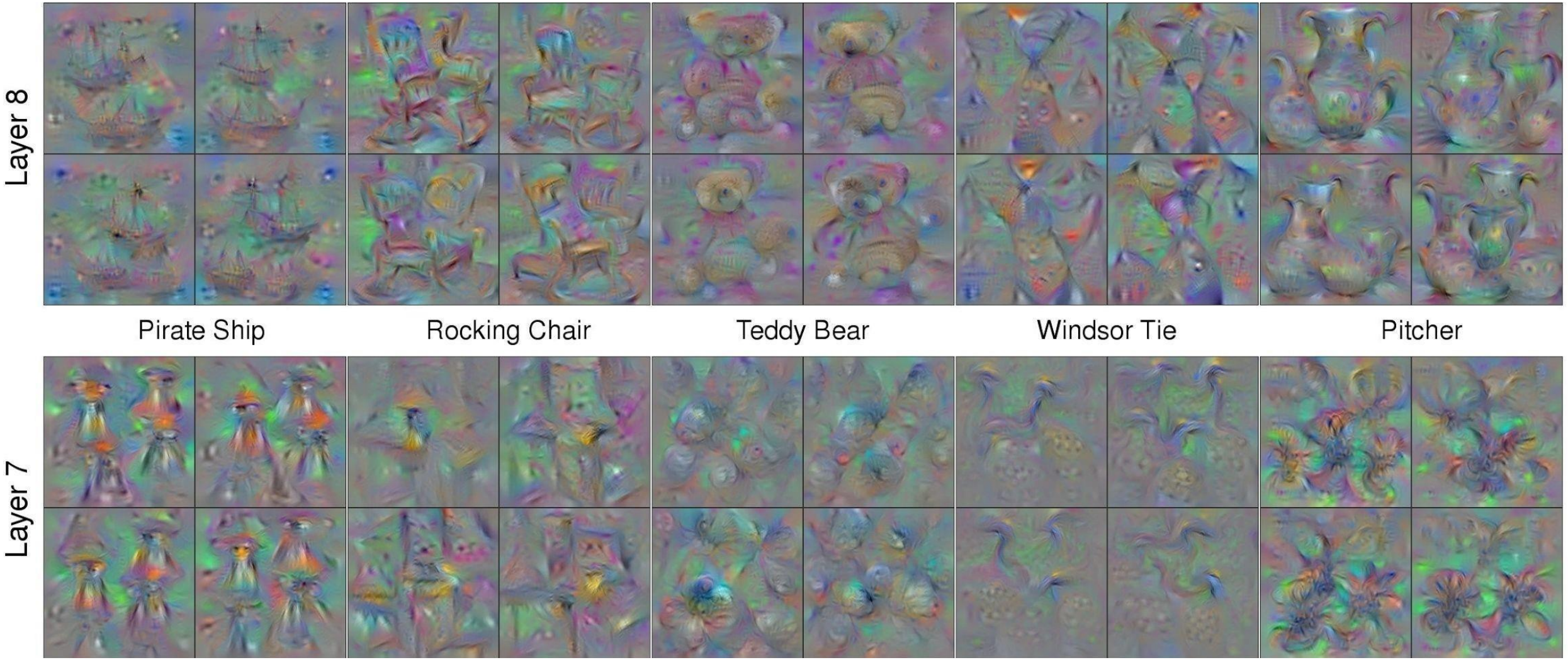
Station Wagon



Black Swan

[Understanding Neural Networks Through Deep Visualization, Yosinski et al. , 2015] http://yosinski.com/deepvis

What image maximizes a class score?



Andrej Karpathy

Saliency Maps

Use backpropagation to compute derivative of classification wrt. each pixel.



Useful to find areas where changes would affect result.

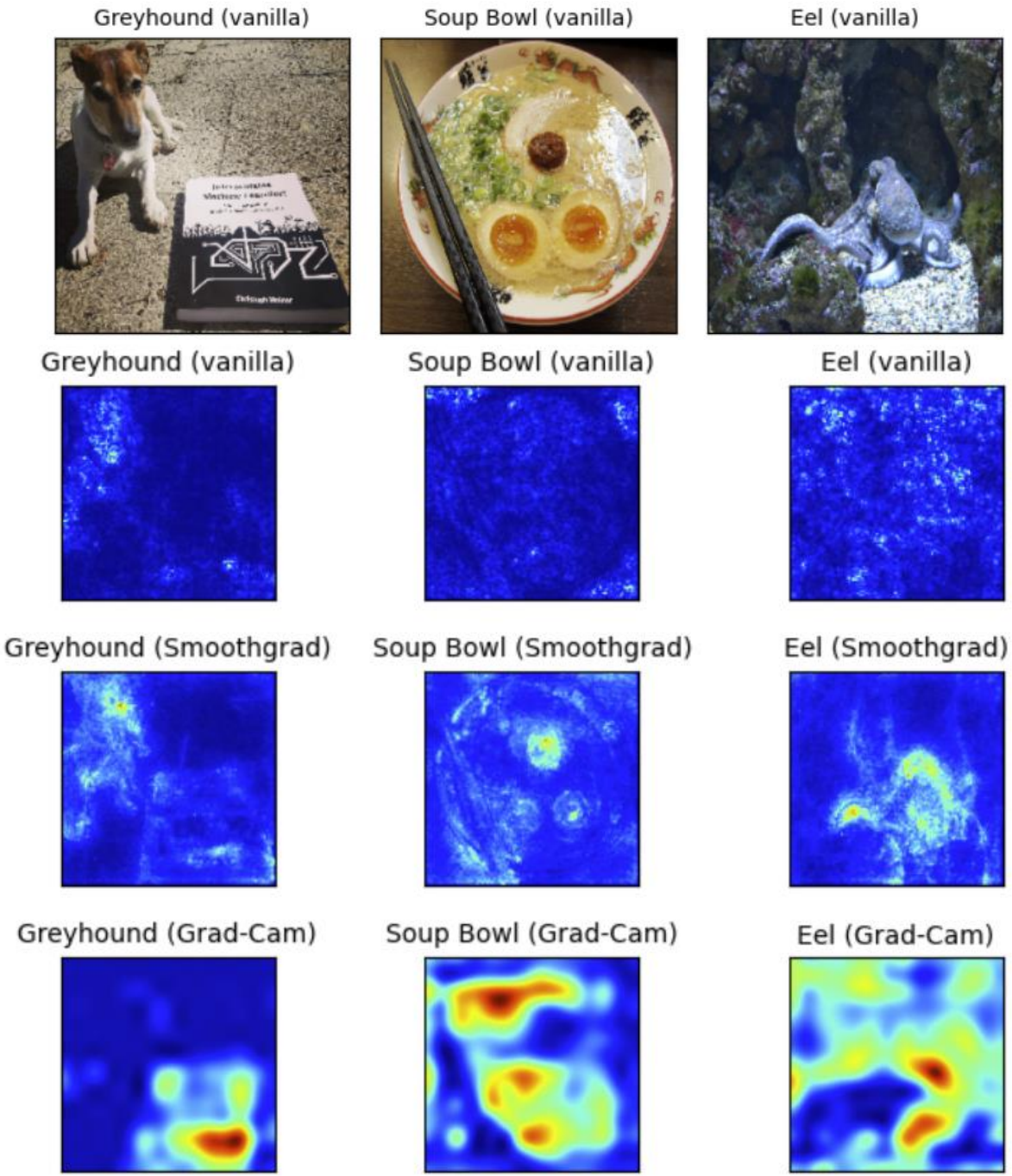
…but outputs can be hard to interpret, especially with multiple scene or object parts.

ImageNet classes of dog, gibbon

[Simonyan et al., 2013]

Output can be hard to interpret.

Varies significantly depending on which method we use.



[Christoph Molnar]

DeepDream

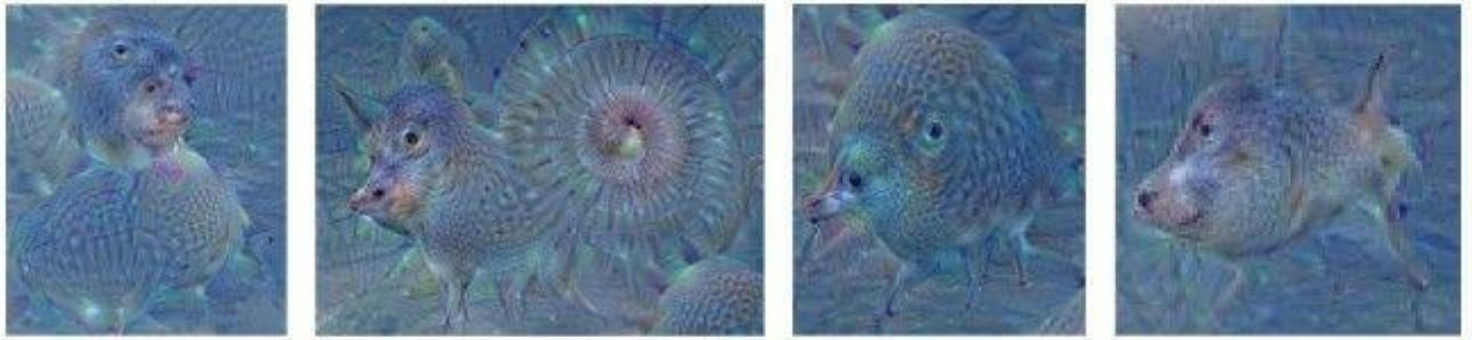


DeepDream https://github.com/google/deepdream

DeepDream – add gradient back to image



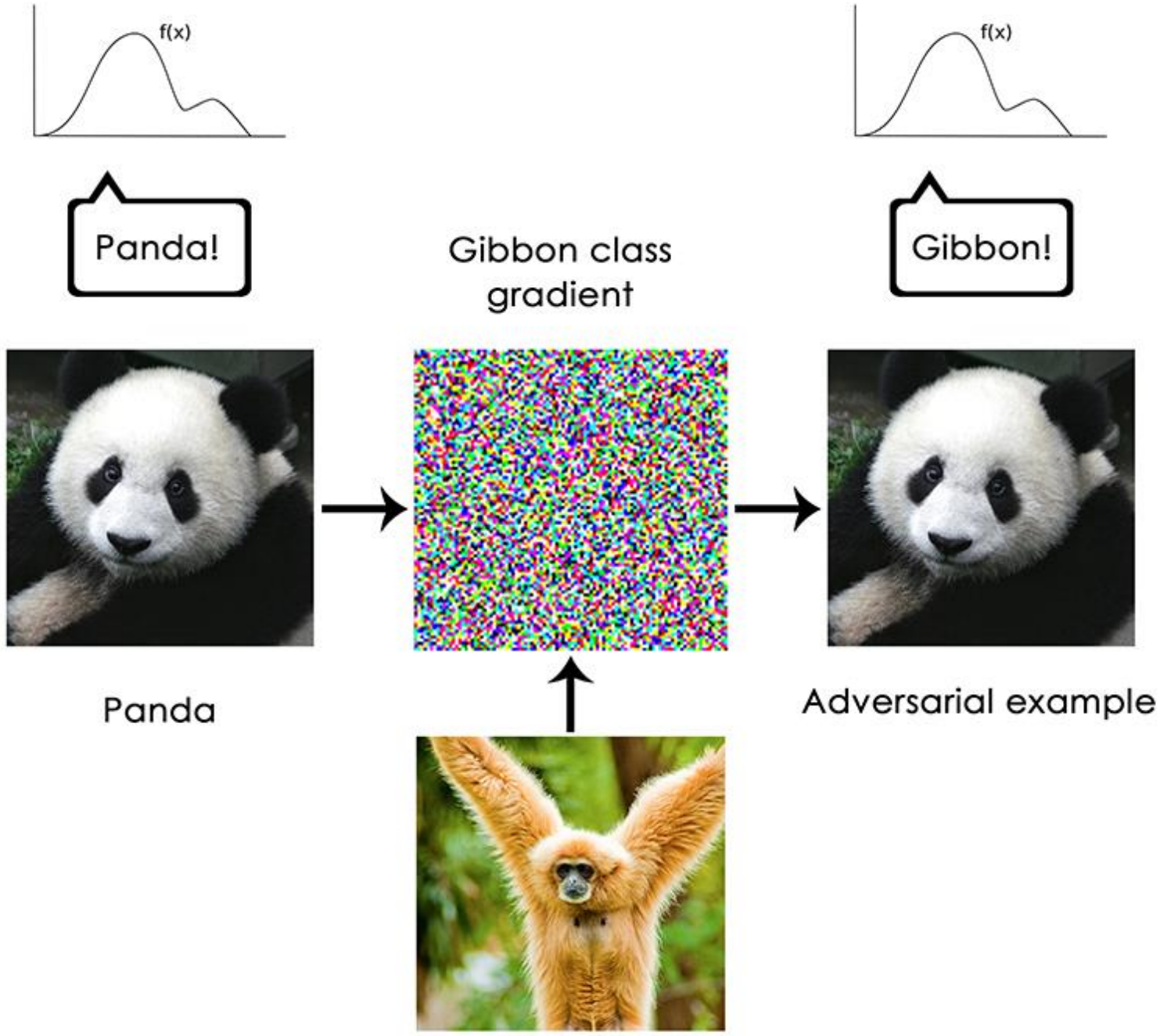
Creates a *feedback loop*: any slightly detected dog face will be made more and more dog-like over time.



DeepDream

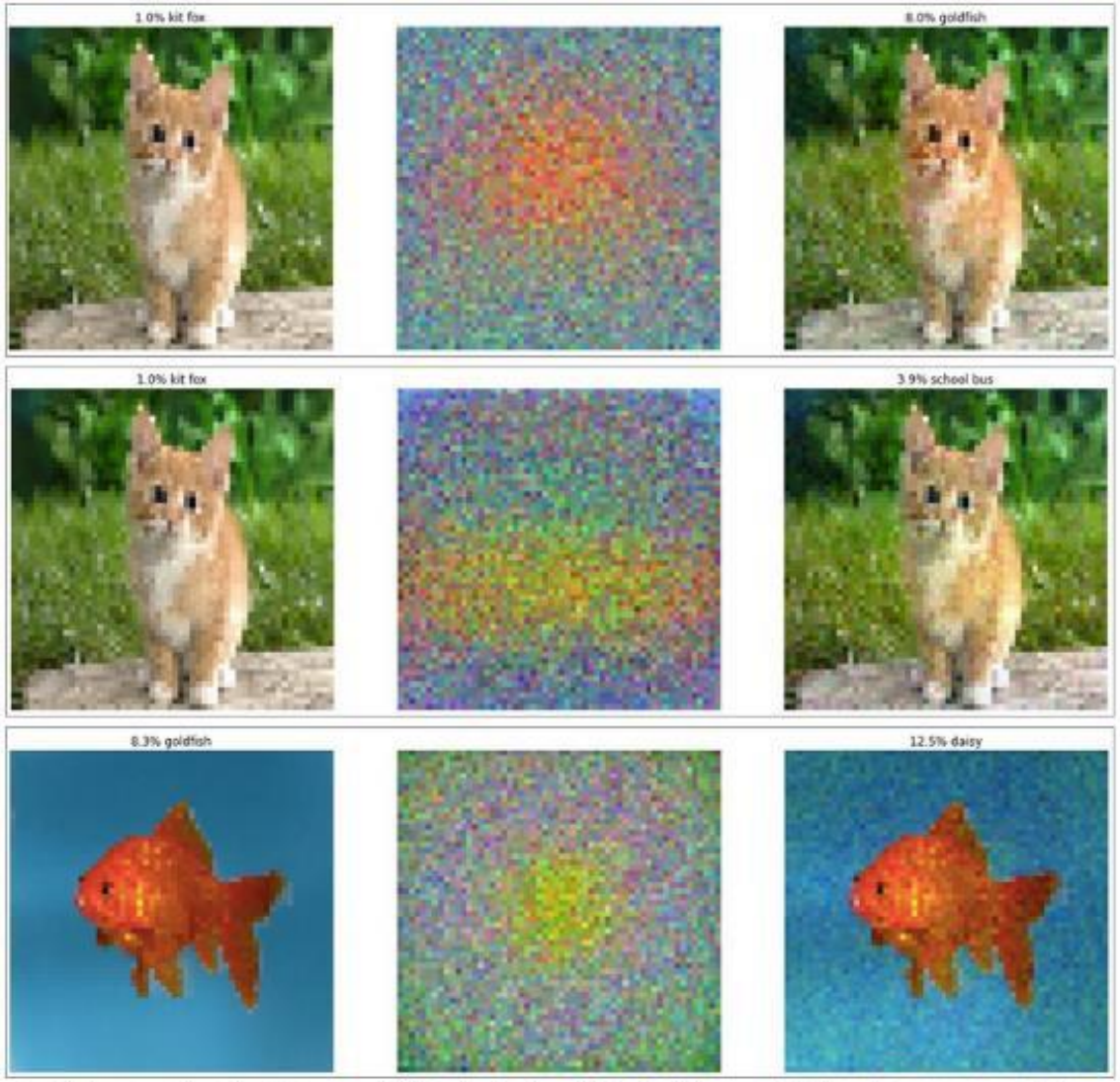


Deep Dream Grocery Trip - https://www.youtube.com/watch?v=DqPaCWJL7XI



Francois Chollet - https://blog.keras.io/the-limitations-of-deep-learning.html

Fooling a linear classifier



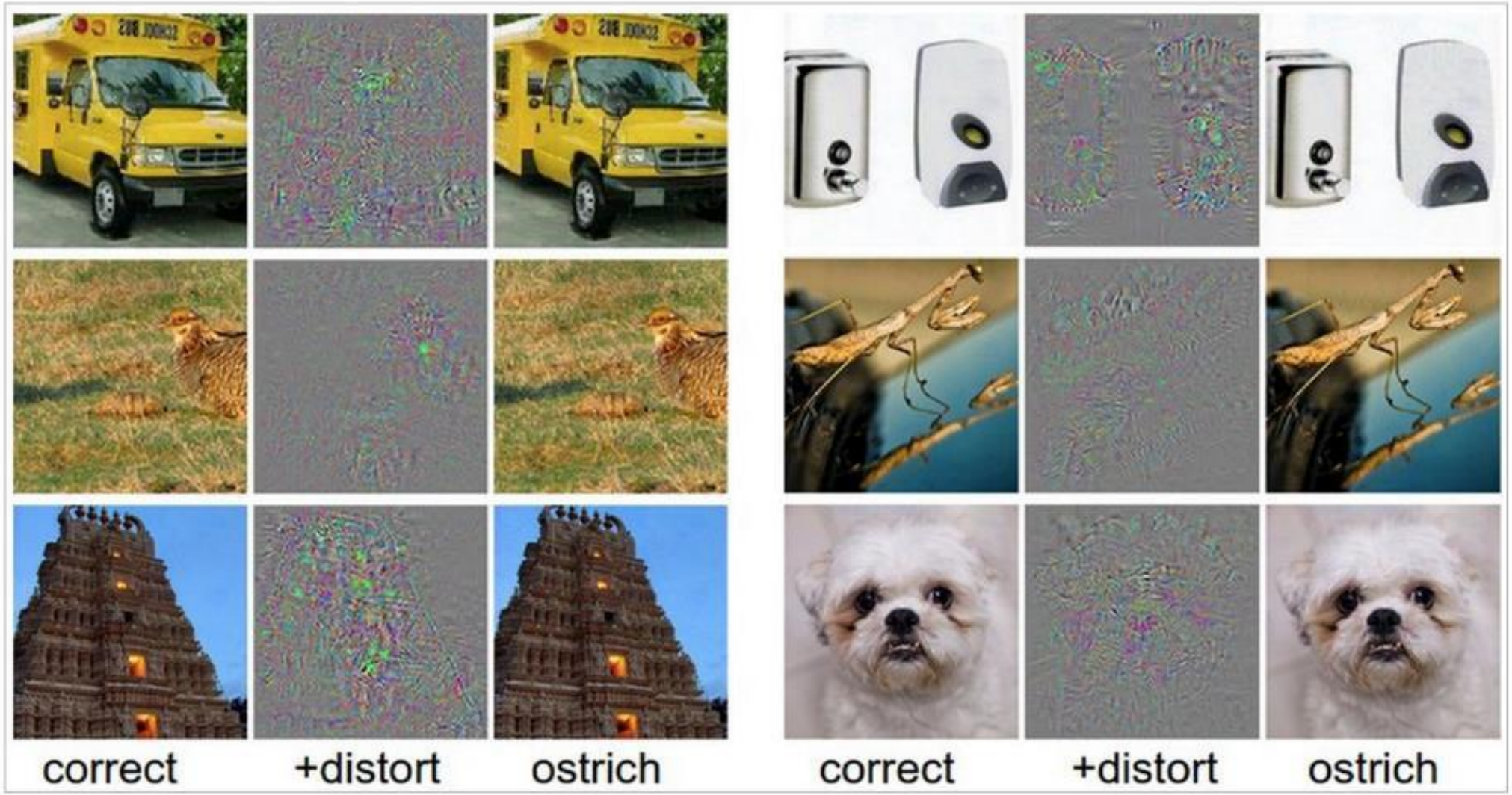
Fooled linear classifier: The starting image (left) is classified as a kit fox. That's incorrect, but then what can you expect from a linear classifier? However, if we add a small amount "goldfish" weights to the image (top row, middle), suddenly the classifier is convinced that it's looking at one with high confidence. We can distort it with the school bus template instead if we wanted to.

To fool a linear classifier, add a small multiple of the weight vector to the training example:

$\mathbf{x} \leftarrow \mathbf{x} + \alpha\mathbf{w}$

http://karpathy.github.io/

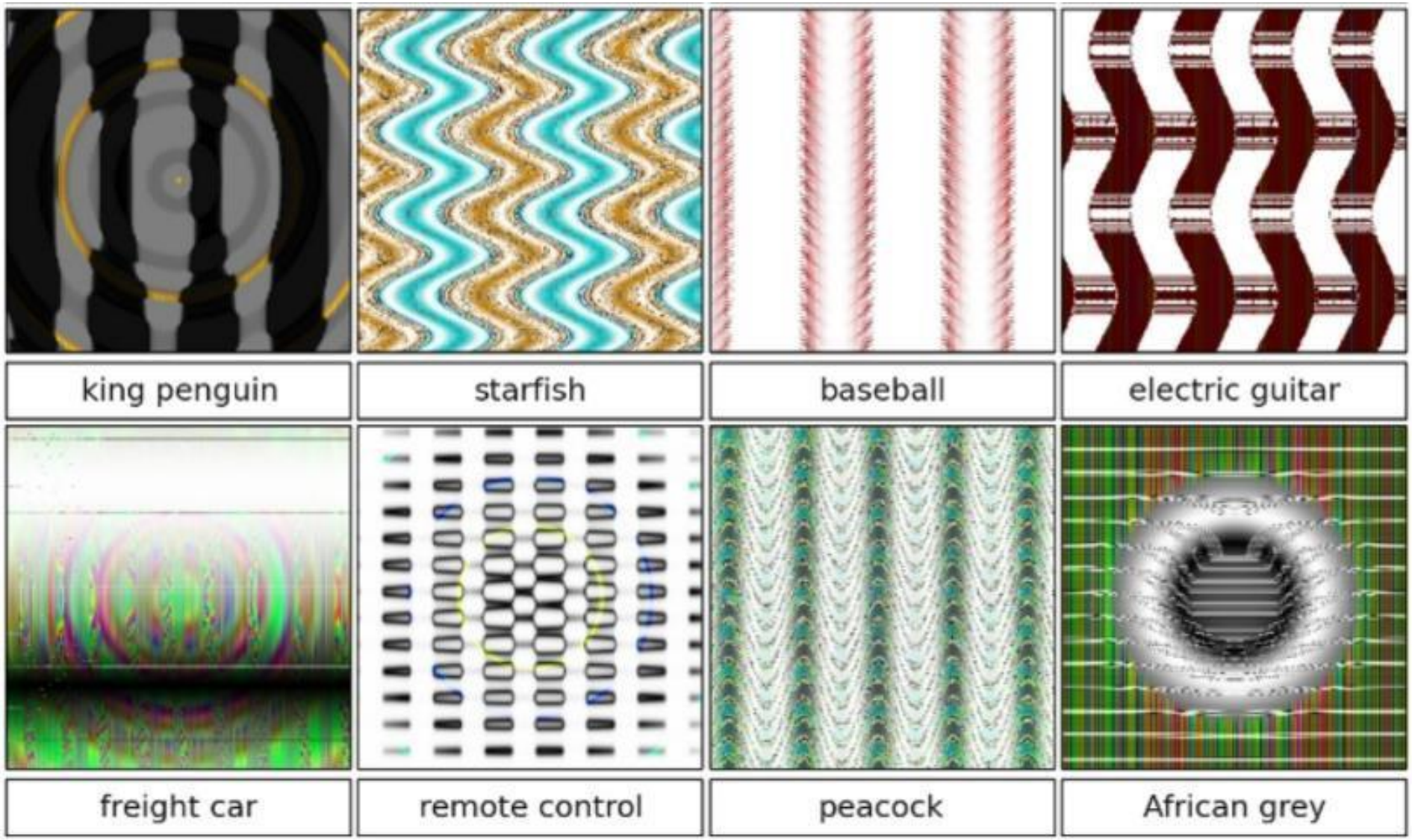
Breaking CNNs



Take a correctly classified image (left image in both columns), and add a tiny distortion (middle) to fool the ConvNet with the resulting image (right).

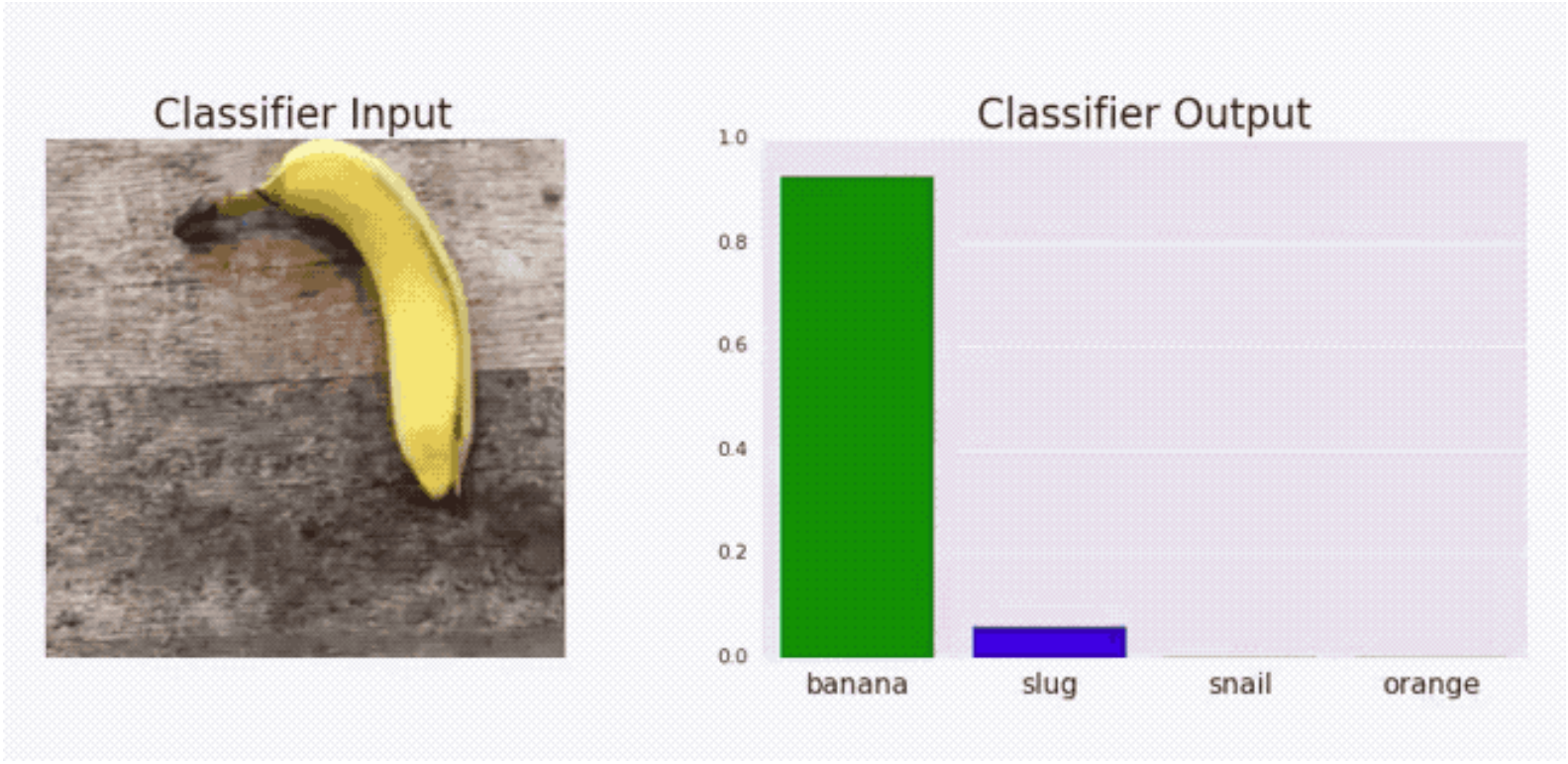
Intriguing properties of neural networks [Szegedy ICLR 2014]

Breaking CNNs



Deep Neural Networks are Easily Fooled: High Confidence Predictions for Unrecognizable Images [Nguyen et al. CVPR 2015]

Adversarial Patches

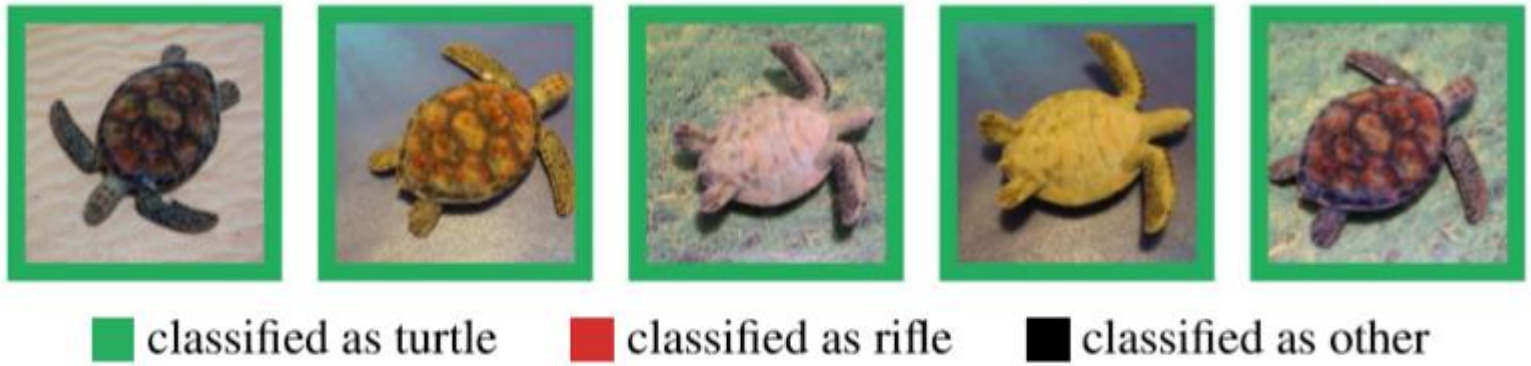


[Brown et al., Adversarial Patch, 2018 - https://arxiv.org/pdf/1712.09665.pdf]

SYNTHESIZING ROBUST ADVERSARIAL EXAMPLES

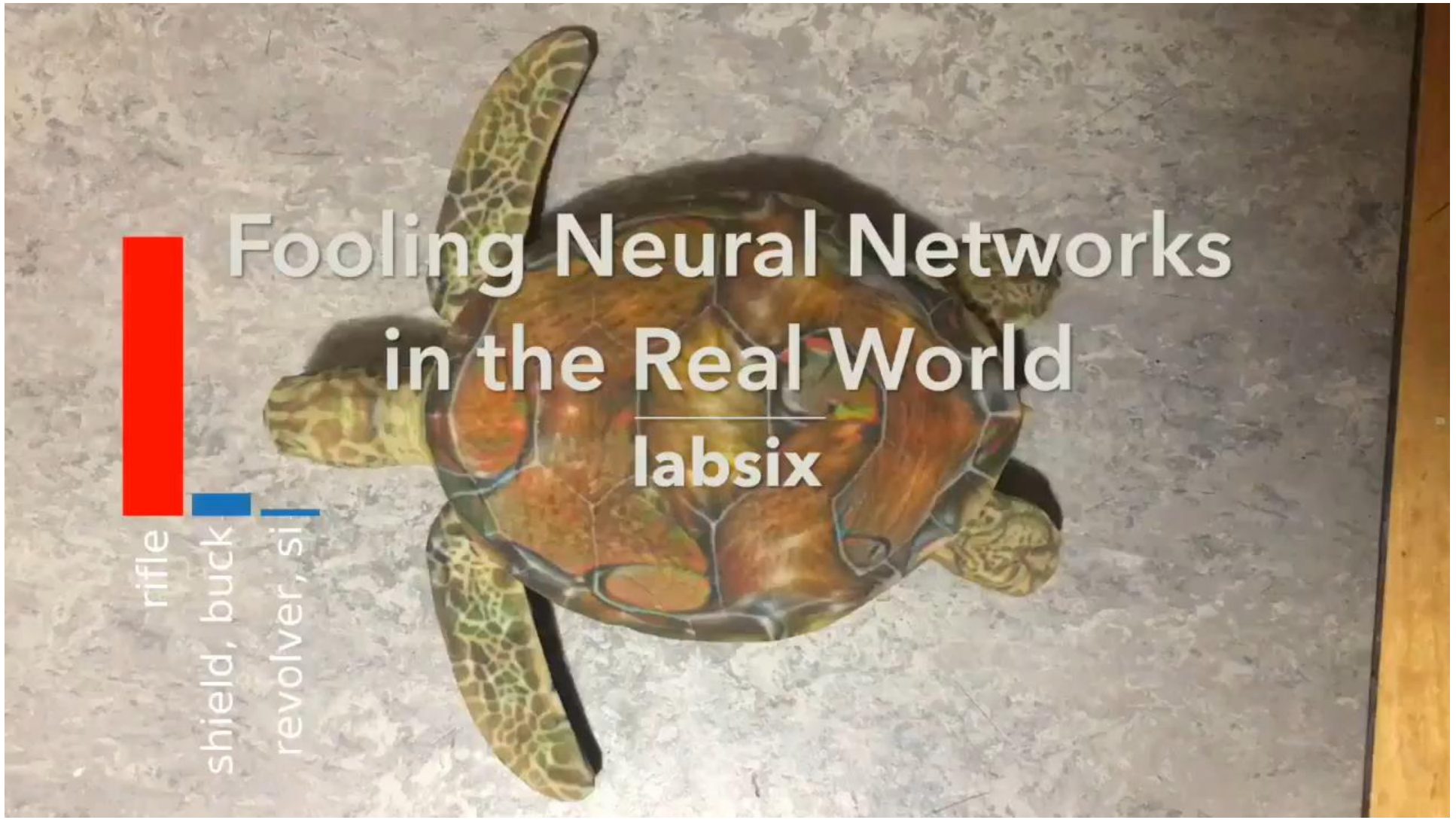
Anish Athalye*[1,2], Logan Engstrom*[1,2], Andrew Ilyas*[1,2], Kevin Kwok[2]

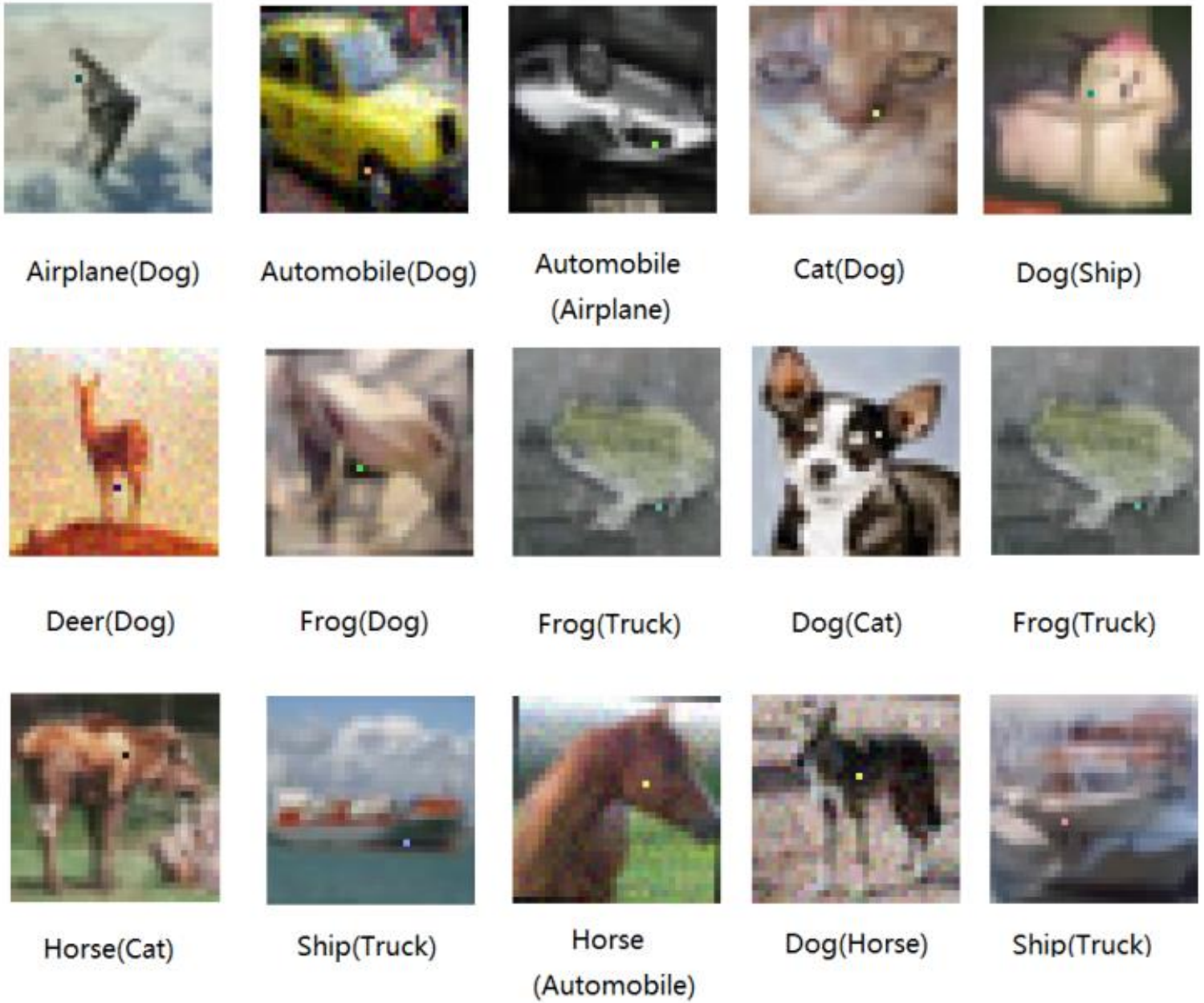
kevin@labsix.org





https://arxiv.org/pdf/1707.07397.pdf


Fooling Neural Networks
in the Real World
labsix
rifle
shield, buck
revolver, si



Su et al., One pixel attack for fooling deep neural networks https://arxiv.org/abs/1710.08864

Adversarial Preprocessing

Erwin Quiring, David Klein, Daniel Arp, Martin Johns and Konrad Rieck

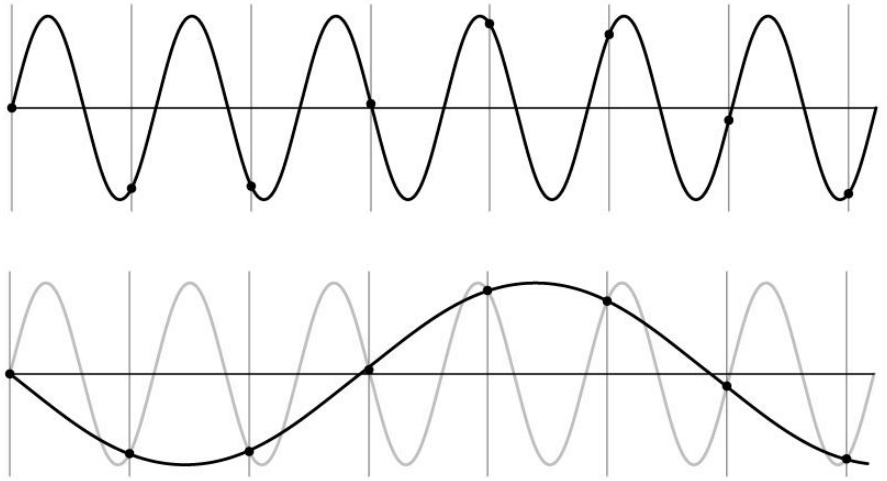
Technische Universität Braunschweig, Germany



Manipulated image Output image

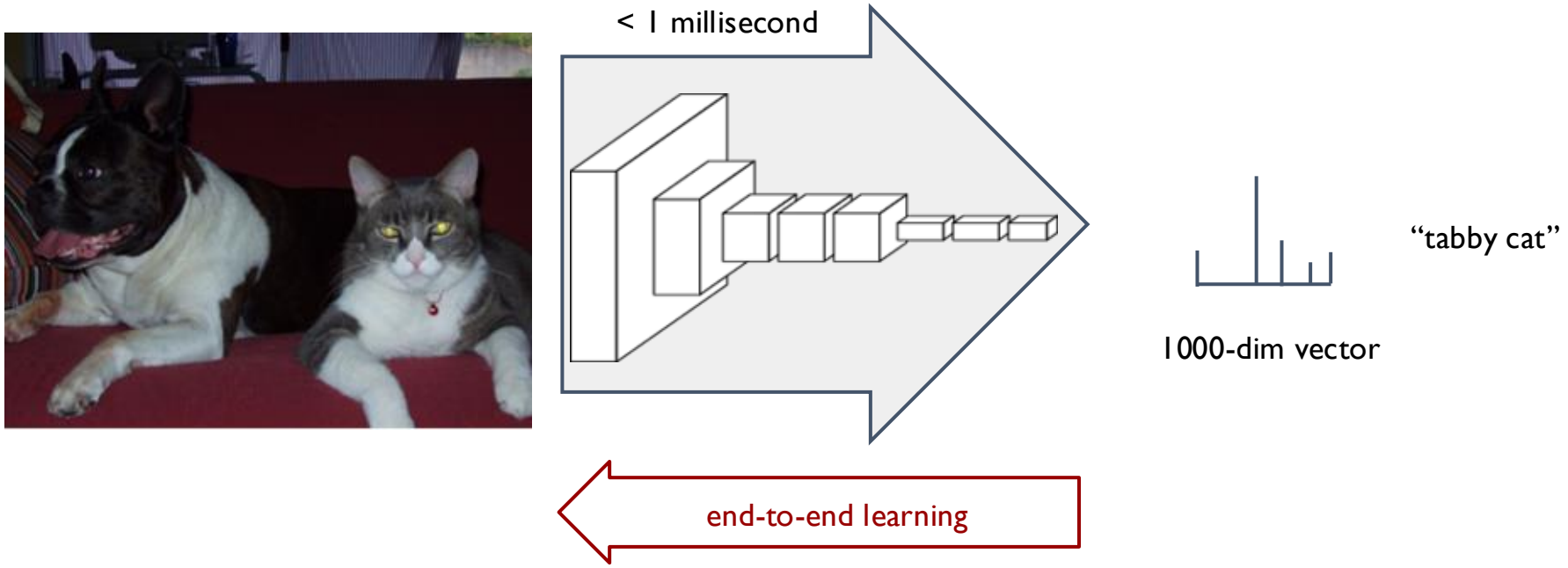
Figure 1: Example of an image-scaling attack. Left: a manipulated image showing a cat. The scaling operation produces the right image with a dog.

Aliasing attack! Downsampling operation does not anti-alias properly.



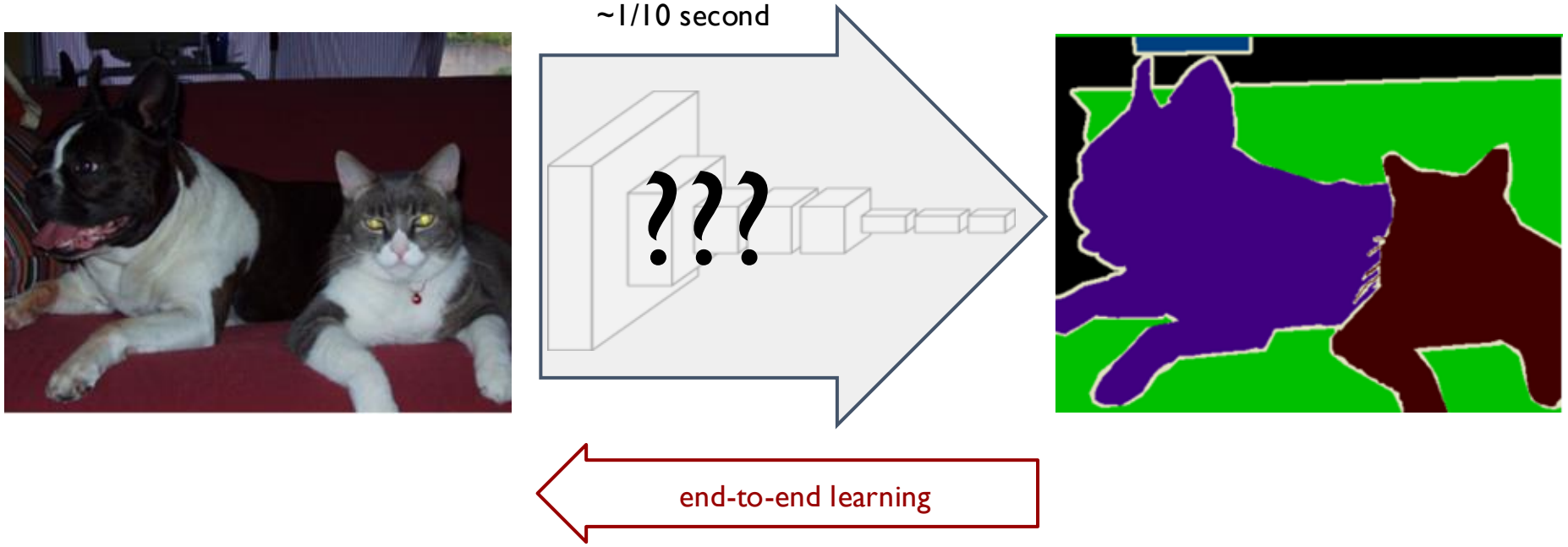
[https://www.usenix.org/system/files/sec20fall_quiring_prepub.pdf]

ConvNets perform classification

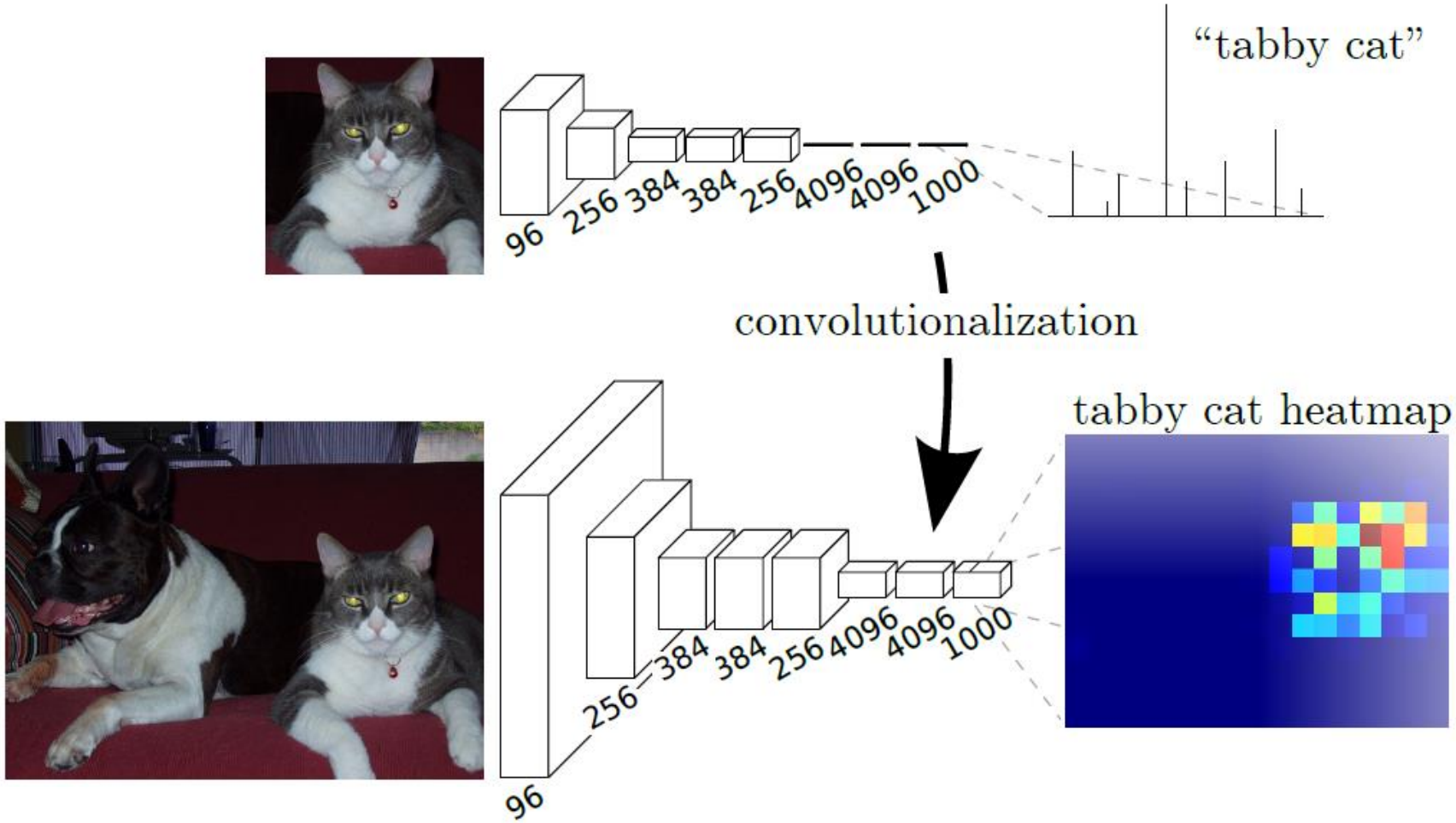


[Slides from Long, Shelhamer, and Darrell; CVPR 2015]

What if we want pixels out?

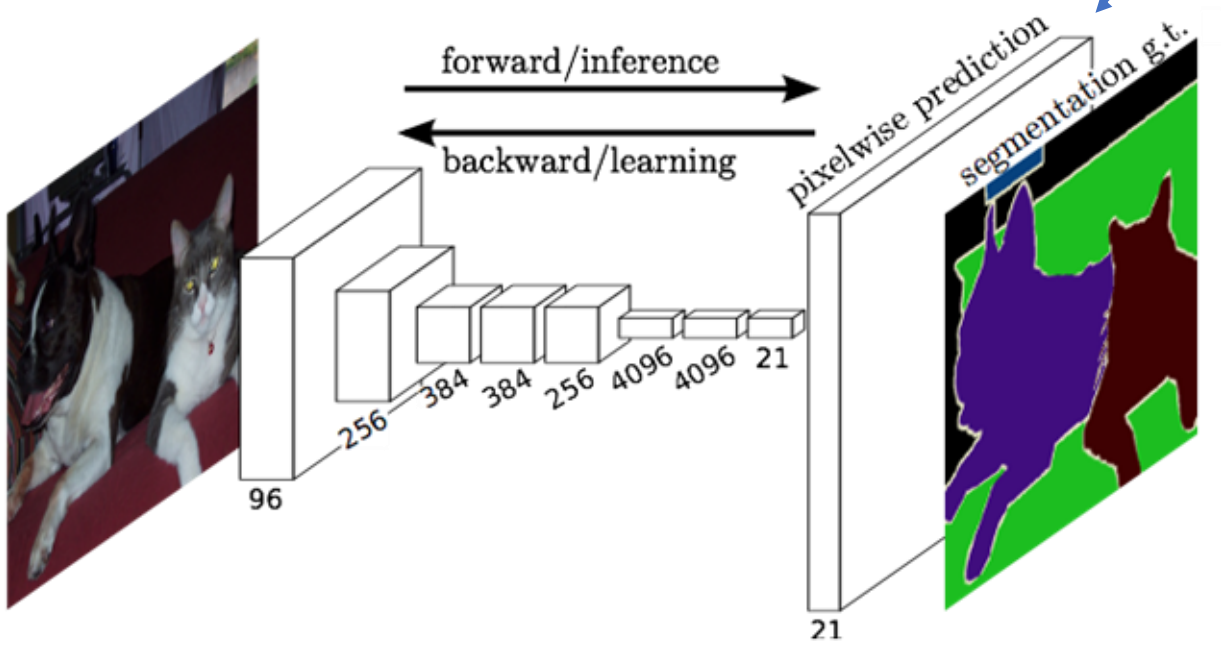


[Slides from Long, Shelhamer, and Darrell; CVPR 2015]


"tabby cat"
96
256
384
384
256
4096
4096
1000
convolutionalization
tabby cat heatmap
96
256
384
384
256
4096
4096
1000

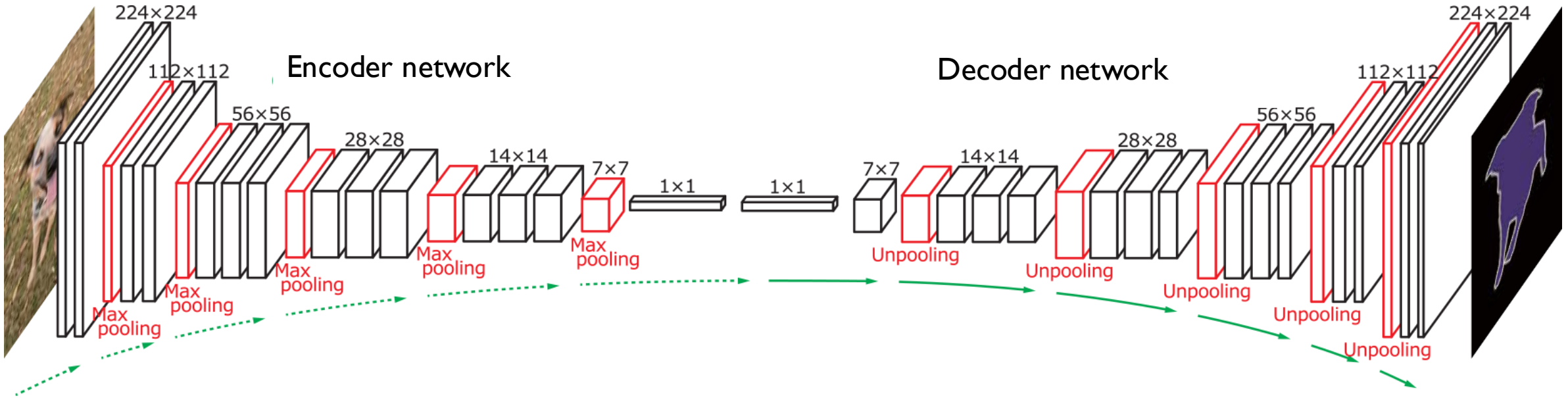
What if we want high-res output?

This one.



Hint: sometimes we use an upsampling *network*

Decoder networks *learn to upsample*



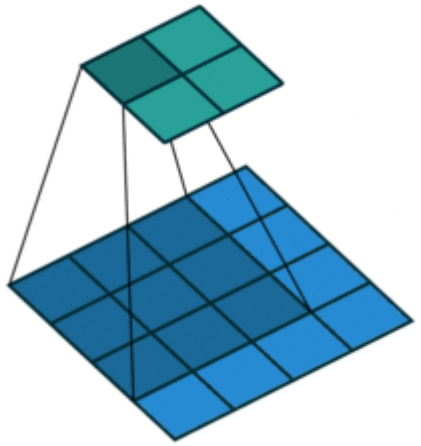
Often called “deconvolution”, but misnomer.
Not the deconvolution that we saw in deblurring -> that is division in the Fourier domain.

‘Transposed convolution’ is better.

Zeiler et al., Deconvolutional Networks, CVPR 2010
Noh et al., Learning Deconvolution Network for Semantic Segmentation, ICCV 2015

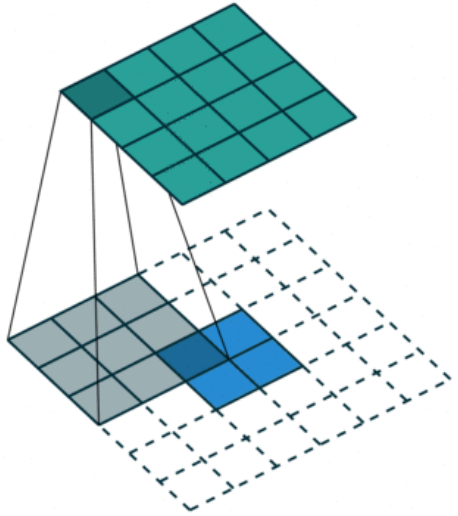
Upsampling with transposed convolution

Convolution



Padding = 0
Stride = 1

Transpose convolution



Padding = 0
Stride = 1

Convolution (sliding window)

Image

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

*

Kernel

1	1	1
1	1	1
1	1	1

=

Result

54	61
90	99

No 1/9 term; not a blur.

Convolution (sliding window)

Image

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

$*$

Kernel

1	1	1
1	1	1
1	1	1

$=$

Result

54	61
90	99

Unroll: Convolution Matrix C

1	1	1	0	1	1	1	0	1	1	1	0	0	0	0	0

$\times$

Vectorized Image

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16

$=$

Result (row vector)^T

54

Convolution (sliding window)

Image

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

$*$

Kernel

1	1	1
1	1	1
1	1	1

$=$

Result

54	61
90	99

Unroll: Convolution Matrix C

1	1	1	0	1	1	1	0	1	1	1	0	0	0	0	0
0	1	1	1	0	1	1	1	0	1	1	1	0	0	0	0

$\times$

Vectorized Image

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16

$=$

Result (row vector)^T

54
61

Convolution (sliding window)

Image

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

$*$

Kernel

1	1	1
1	1	1
1	1	1

$=$

Result

54	61
90	99

Unroll: Convolution Matrix C

1	1	1	0	1	1	1	0	1	1	1	0	0	0	0	0
0	1	1	1	0	1	1	1	0	1	1	1	0	0	0	0
0	0	0	0	1	1	1	0	1	1	1	0	1	1	1	0

$\times$

Vectorized Image

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16

$=$

Result (row vector)^T

54
61
90

Convolution (sliding window)

Image

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

$*$

Kernel

1	1	1
1	1	1
1	1	1

$=$

Result

54	61
90	99

Unroll: Convolution Matrix C

1	1	1	0	1	1	1	0	1	1	1	0	0	0	0	0
0	1	1	1	0	1	1	1	0	1	1	1	0	0	0	0
0	0	0	0	1	1	1	0	1	1	1	0	1	1	1	0
0	0	0	0	0	1	1	1	0	1	1	1	0	1	1	1

$\times$

Vectorized Image

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16

$=$

Result (row vector)^T

54
61
90
99

Transposed Convolution Matrix

C^T

1	0	0	0
1	1	0	0
1	1	0	0
0	1	0	0
1	0	1	0
1	1	1	1
1	1	1	1
0	1	0	1
1	0	1	0
1	1	1	1
1	1	1	1
0	1	0	1
0	0	1	0
0	0	1	1
0	0	1	1
0	0	0	1

×

"Image"

1
2
3
4

2 x 2

=

Result

1
3
3
2
4
10
10
6
4
10
10
6
3
7
7
4

Result (re-formed)

1	3	3	2
4	10	10	6
4	10	10	6
3	7	7	4

4 x 4

Note: this is no longer a convolution operation.

Transpose conv. = backward pass of conv.

Convolution

$$\begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix} * \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} \\ h_{21} & h_{22} \end{bmatrix}$$

$$h_{11} = x_{11}w_{11} + x_{12}w_{12} + x_{21}w_{21} + x_{22}w_{22}$$
$$h_{12} = x_{12}w_{11} + x_{13}w_{12} + x_{22}w_{21} + x_{23}w_{22}$$
$$h_{21} = x_{21}w_{11} + x_{22}w_{12} + x_{31}w_{21} + x_{32}w_{22}$$
$$h_{22} = x_{22}w_{11} + x_{23}w_{12} + x_{32}w_{21} + x_{33}w_{22}$$

Transpose conv. = backward pass of conv.

Convolution

$$\begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix} * \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} \\ h_{21} & h_{22} \end{bmatrix}$$

Derivative of convolution in weight update step

∂W:

$$\delta w_{11} = x_{11}\delta h_{11} + x_{12}\delta h_{12} + x_{21}\delta h_{21} + x_{22}\delta h_{22}$$
$$\delta w_{12} = x_{12}\delta h_{11} + x_{13}\delta h_{12} + x_{22}\delta h_{21} + x_{23}\delta h_{22}$$
$$\delta w_{21} = x_{21}\delta h_{11} + x_{22}\delta h_{12} + x_{31}\delta h_{21} + x_{32}\delta h_{22}$$
$$\delta w_{22} = x_{22}\delta h_{11} + x_{23}\delta h_{12} + x_{32}\delta h_{21} + x_{33}\delta h_{22}$$

Transpose conv. = backward pass of conv.

Convolution

$$\begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix} * \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} \\ h_{21} & h_{22} \end{bmatrix}$$

∂X:

$$\delta x_{11} = w_{11}\delta h_{11}$$
$$\delta x_{12} = w_{12}\delta h_{11} + w_{11}\delta h_{12}$$
$$\delta x_{13} = w_{12}\delta h_{12}$$
$$\delta x_{21} = w_{21}\delta h_{11} + w_{11}\delta h_{21}$$
$$\delta x_{22} = w_{22}\delta h_{11} + w_{21}\delta h_{12} + w_{12}\delta h_{21} + w_{11}\delta h_{22}$$
$$\delta x_{23} = w_{22}\delta h_{12} + w_{12}\delta h_{22}$$
$$\delta x_{31} = w_{21}\delta h_{21}$$
$$\delta x_{32} = w_{22}\delta h_{21} + w_{21}\delta h_{22}$$
$$\delta x_{33} = w_{22}\delta h_{22}$$

Transpose conv. = backward pass of conv.

Convolution

$$\begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix} * \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} \\ h_{21} & h_{22} \end{bmatrix}$$

∂X:

$$\delta x_{11} = w_{11}\delta h_{11}$$
$$\delta x_{12} = w_{12}\delta h_{11} + w_{11}\delta h_{12}$$
$$\delta x_{13} = w_{12}\delta h_{12}$$
$$\delta x_{21} = w_{21}\delta h_{11} + w_{11}\delta h_{21}$$
$$\delta x_{22} = w_{22}\delta h_{11} + w_{21}\delta h_{12} + w_{12}\delta h_{21} + w_{11}\delta h_{22}$$
$$\delta x_{23} = w_{22}\delta h_{12} + w_{12}\delta h_{22}$$
$$\delta x_{31} = w_{21}\delta h_{21}$$
$$\delta x_{32} = w_{22}\delta h_{21} + w_{21}\delta h_{22}$$
$$\delta x_{33} = w_{22}\delta h_{22}$$

Operations are the same!

$C^\top$

w_{11}	0	0	0
w_{12}	w_{11}	0	0
0	w_{12}	0	0
w_{21}	0	w_{11}	0
w_{22}	w_{21}	w_{12}	w_{11}
0	w_{22}	0	w_{12}
0	0	w_{21}	0
0	0	w_{22}	w_{21}
0	0	0	w_{22}

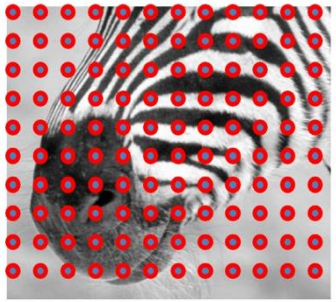
$\times$

δh_{11}
δh_{12}
δh_{21}
δh_{22}

Analog to upsampling/downsampling

Downsampling (no aliasing):

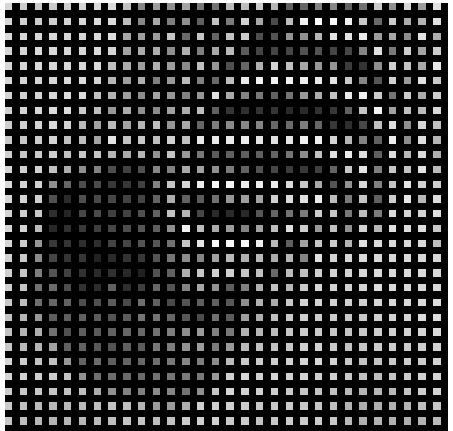
- Blur image (convolve)
- Throw away every second sample along x,y



Upsampling:

- Add zeros between every sample along x,y
- Bilinearly interpolate by convolving with tent kernel



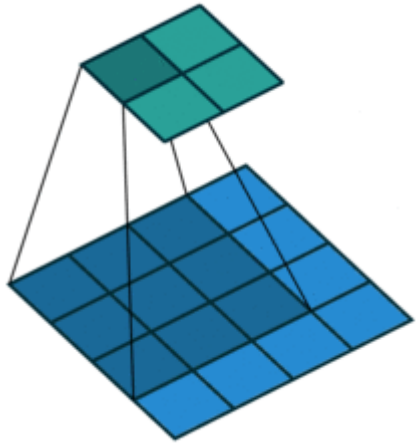


Convolve with bilinear (tent) kernel

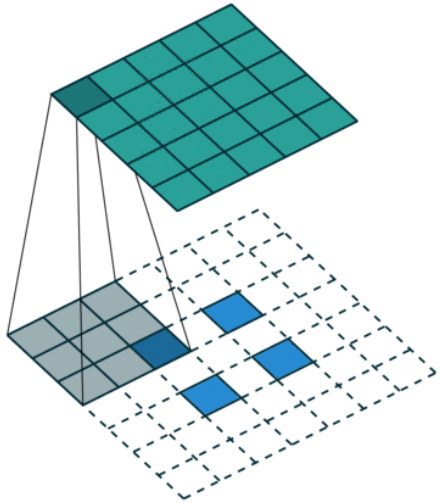


Analog to upsampling/downsampling

Convolution



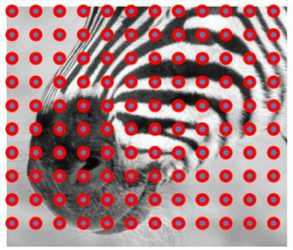
Transposed convolution as fractionally-strided convolution



2x2, stride 2, 3x3 kernel, upsample to 5x5.

Analog to upsampling/downsampling

Downsampling (no aliasing):
- Blur image to remove high frequencies (convolve)
- Throw away every second sample along x,y



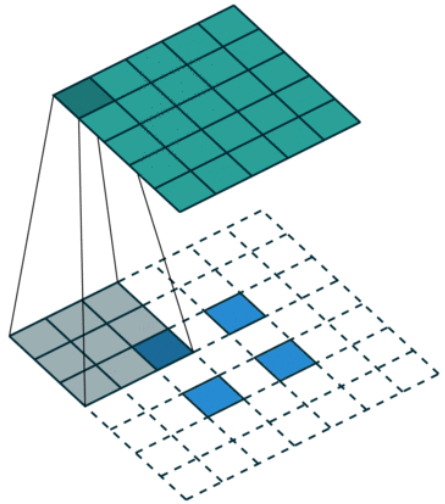
Upsampling:
- Add zeros between every sample along x,y
- Bilinearly interpolate by convolving with tent kernel

Upsampling analogy implies that *strided convolution* can be used to implement transpose convolution.

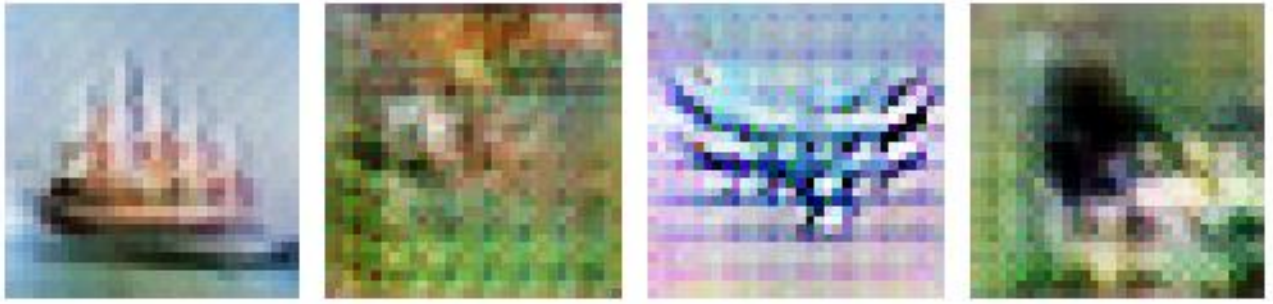
Called '*fractionally-strided convolution*' as we insert empty space (new zero-valued pixels) 'halfway' between pixels.

Kernel size/stride is important. Is uneven overlap a problem?

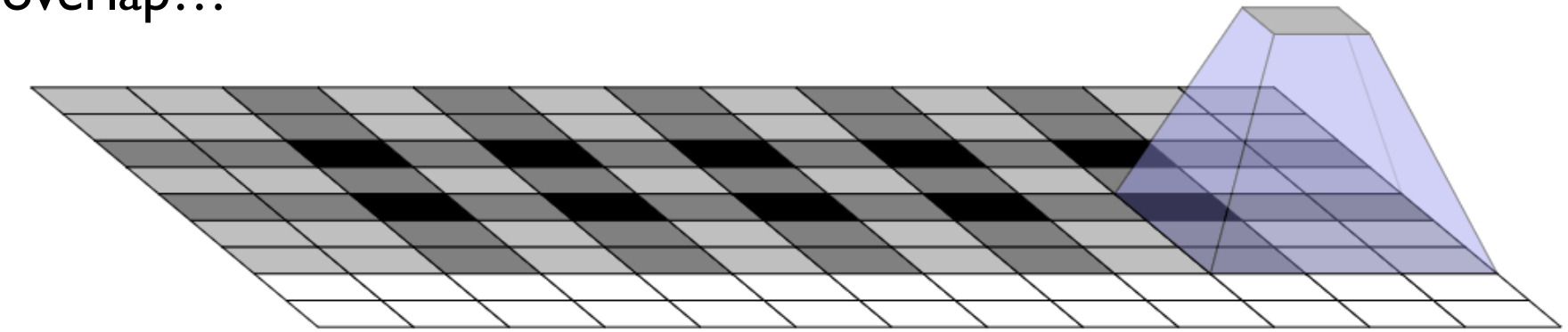
Uneven overlap across output



Yes = causes grid artifacts

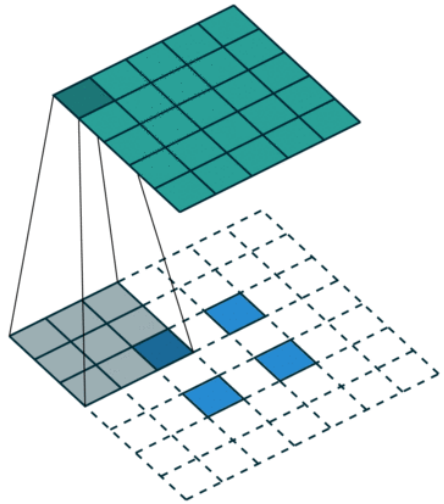


Could fix it by picking stride/kernel numbers which have no overlap…

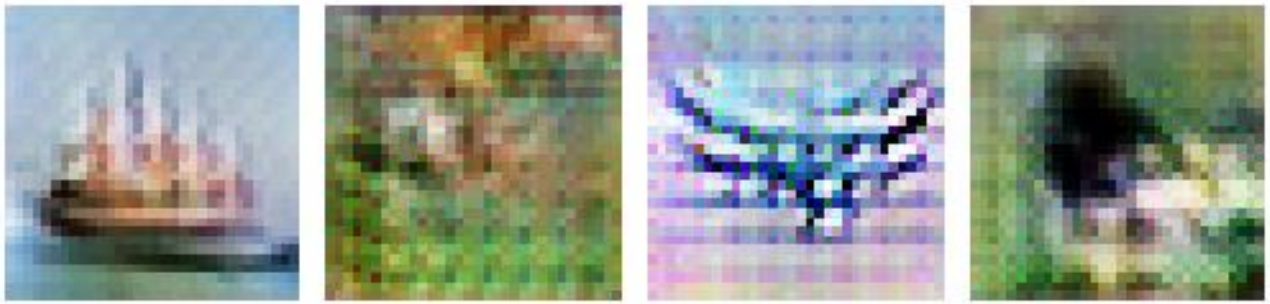


Is uneven overlap a problem?

Uneven overlap across output



Yes = causes grid artifacts

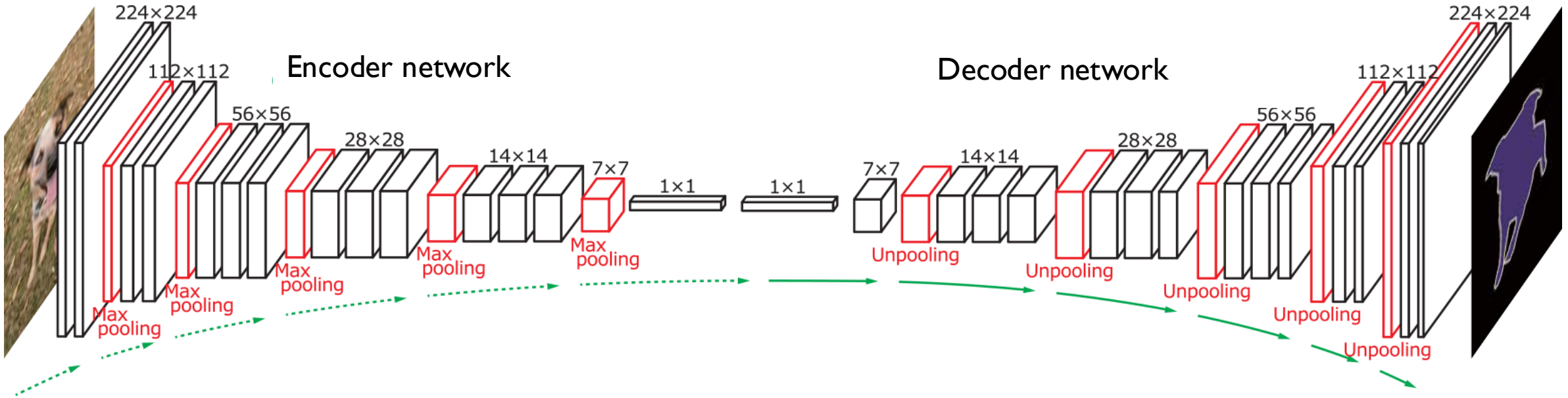


Could fix it by picking stride/kernel numbers which have no overlap…

Or…*think in frequency*!
Introduce explicit bilinear upsampling before transpose convolution; let kernels of transpose convolution learn to fill in only high-frequency detail.

https://distill.pub/2016/deconv-checkerboard/

Decoder networks *learn to upsample*

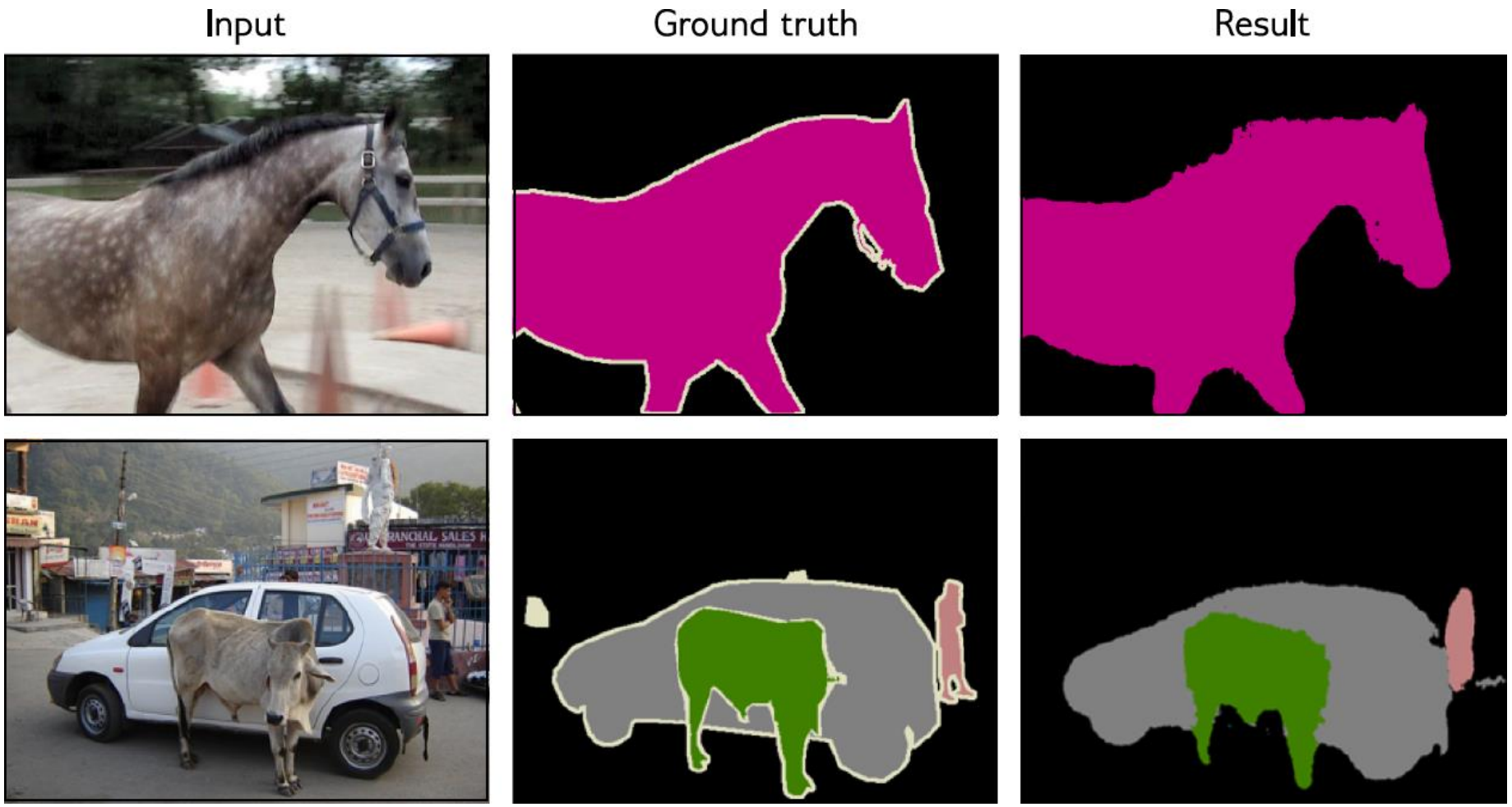


Often called “deconvolution”, but misnomer.
Not the deconvolution that we saw in deblurring -> that is division in the Fourier domain.

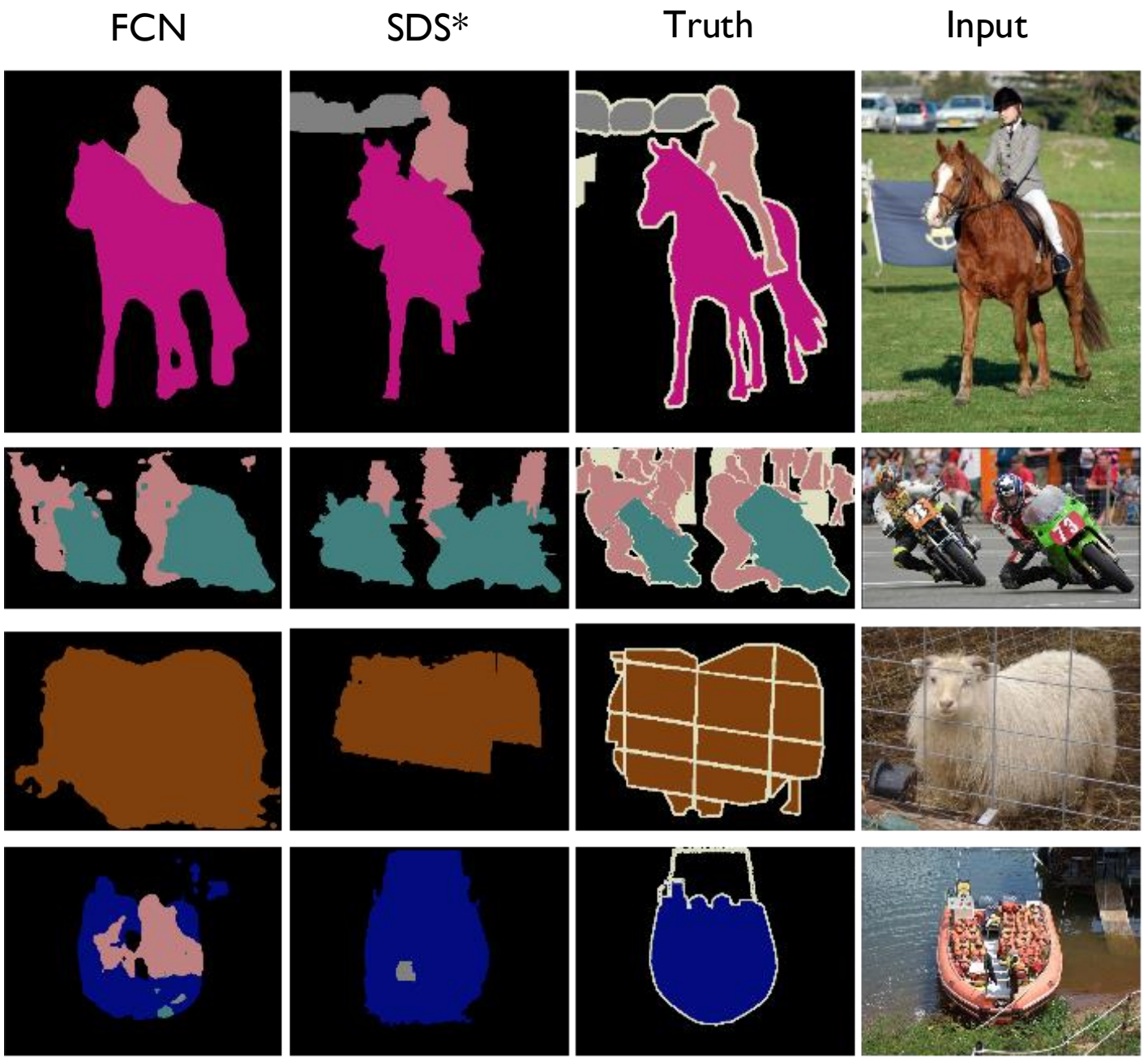
‘Transposed convolution’ is better.

Zeiler et al., Deconvolutional Networks, CVPR 2010
Noh et al., Learning Deconvolution Network for Semantic Segmentation, ICCV 2015

Semantic segmentation results



Results

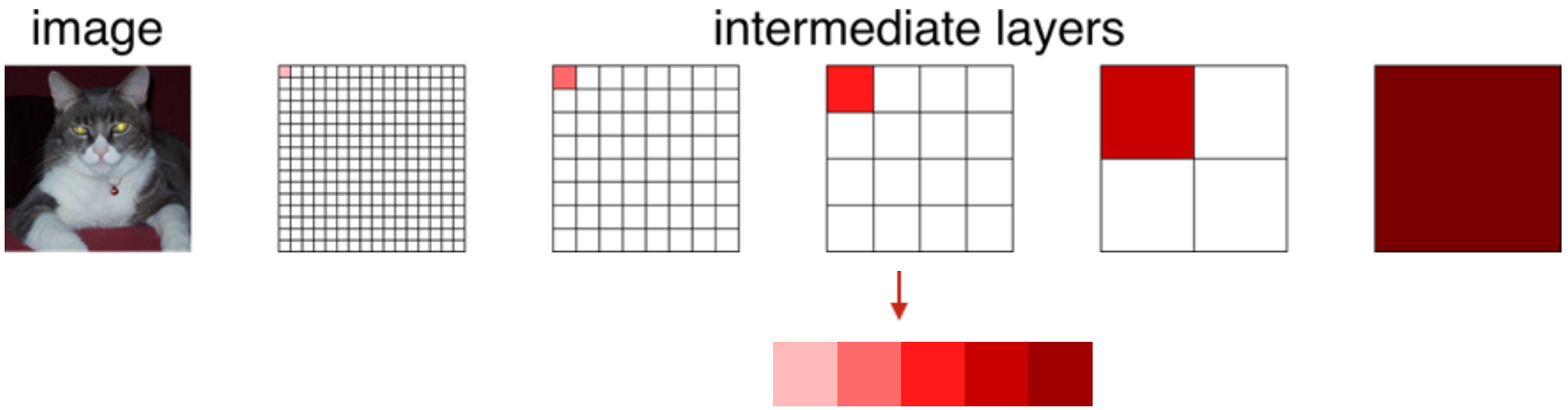


SDS = *Simultaneous Detection and Segmentation Hariharan et al. ECCV14
Not Score Distillation Sampling…

[Long et al.]

Scale-space of deep features (concatenated)

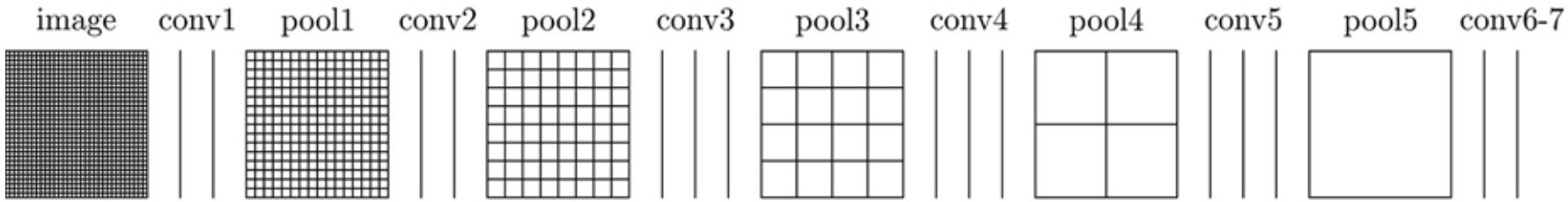
Combine *where* (local, shallow) with *what* (global, deep)



Fuse features into **deep jet**

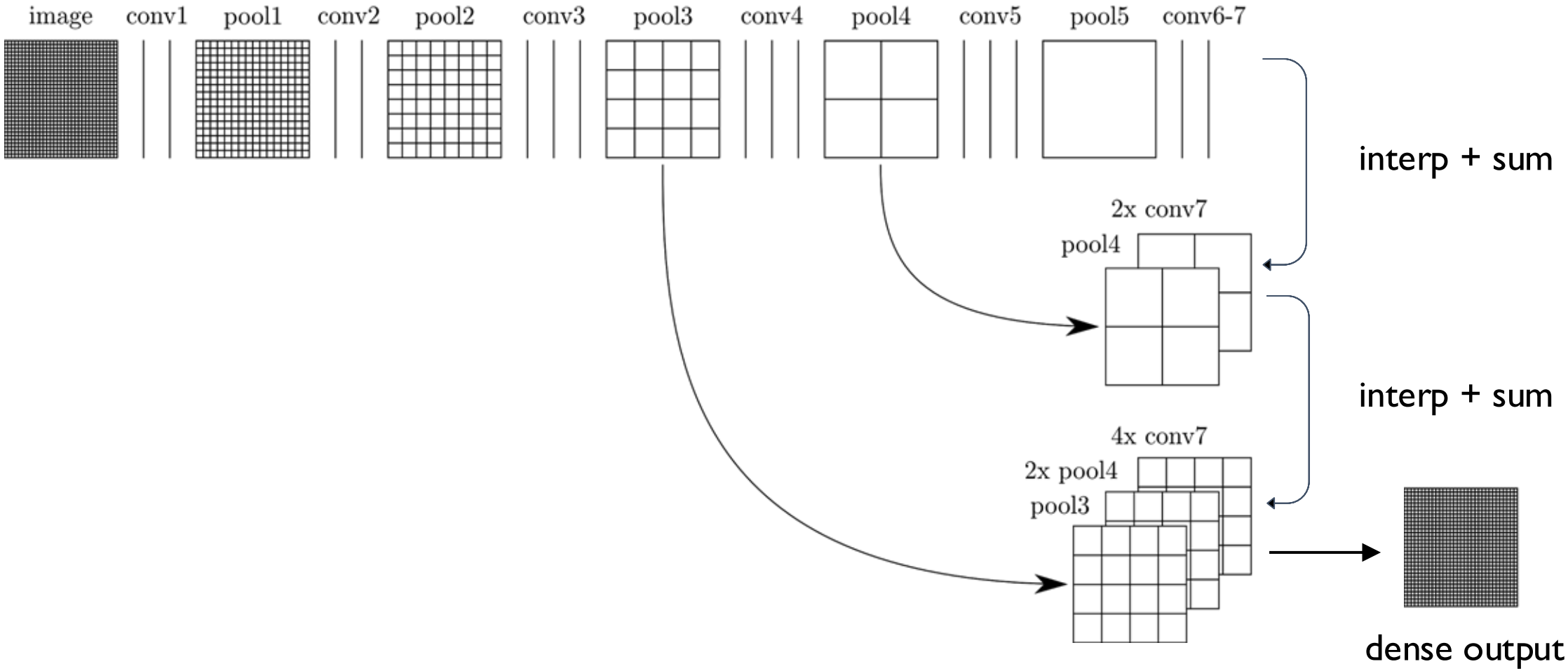
(cf. Hariharan et al. CVPR15 "hypercolumn")

Upsampling kernels with skip layer refinement



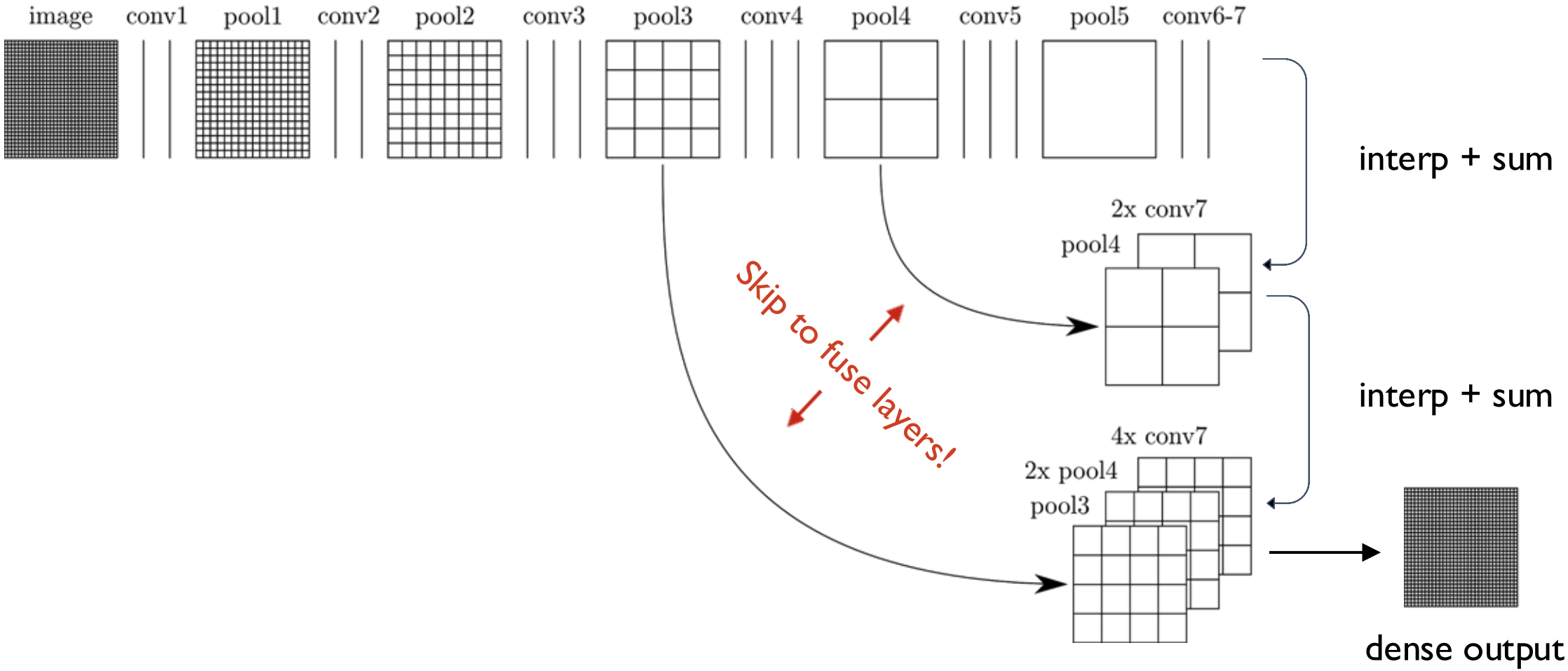
[Long et al.]

Upsampling kernels with skip layer refinement



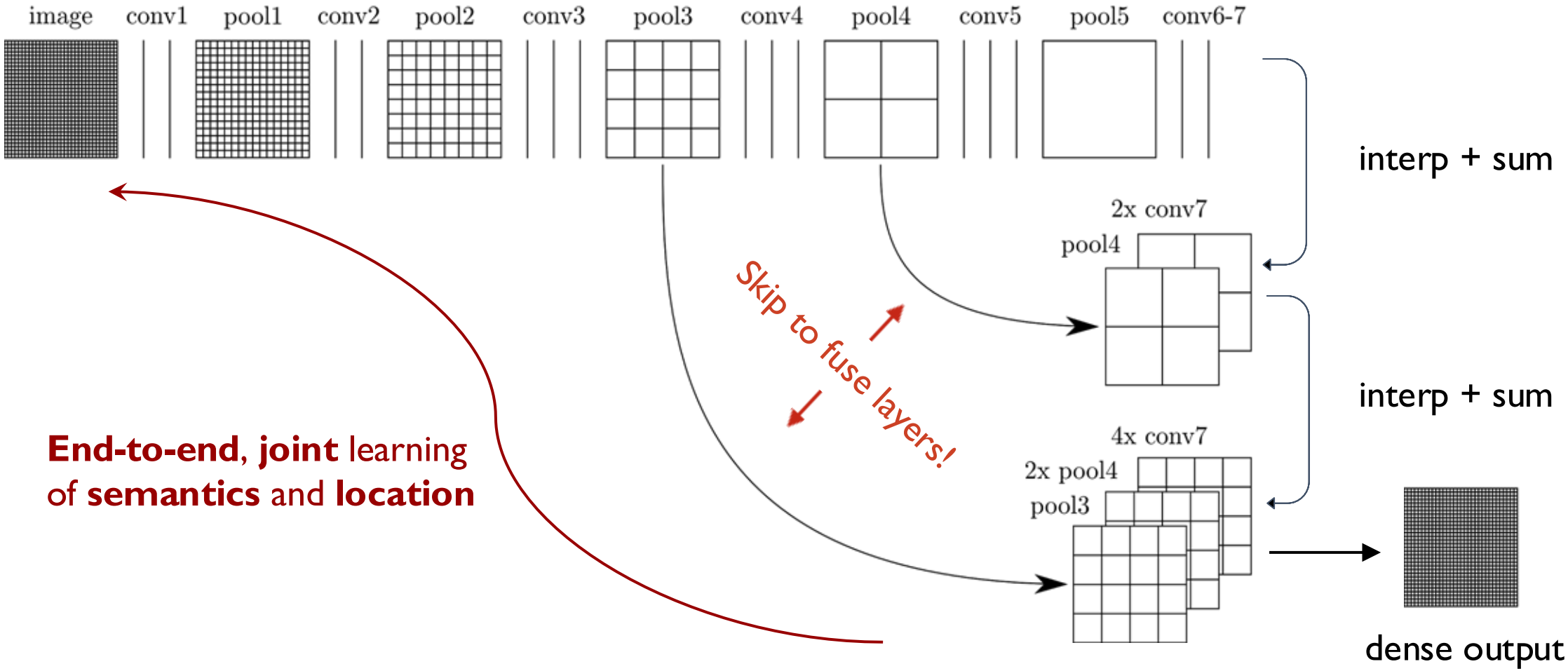
[Long et al.]

Upsampling kernels with skip layer refinement



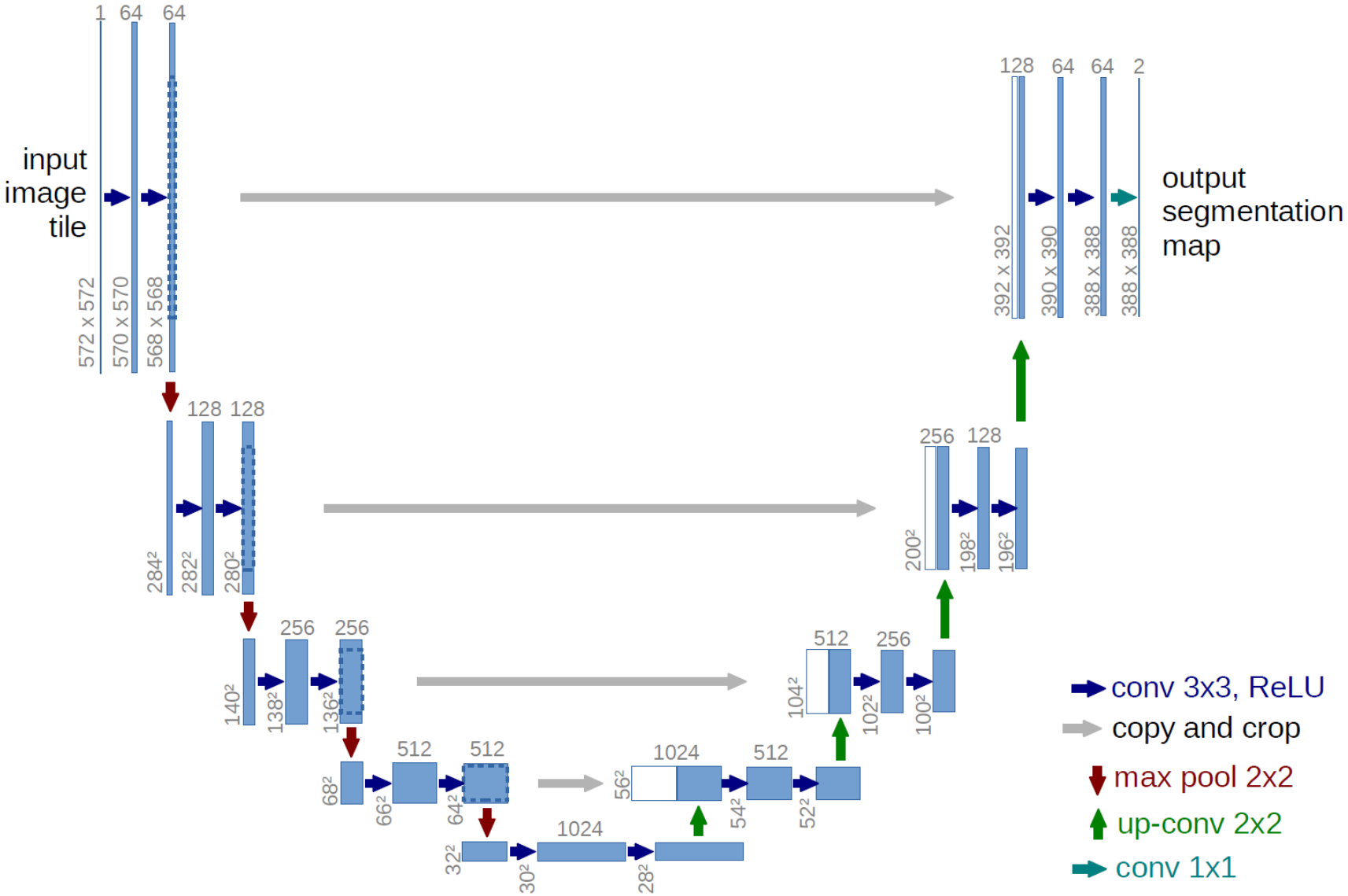
[Long et al.]

Upsampling kernels with skip layer refinement

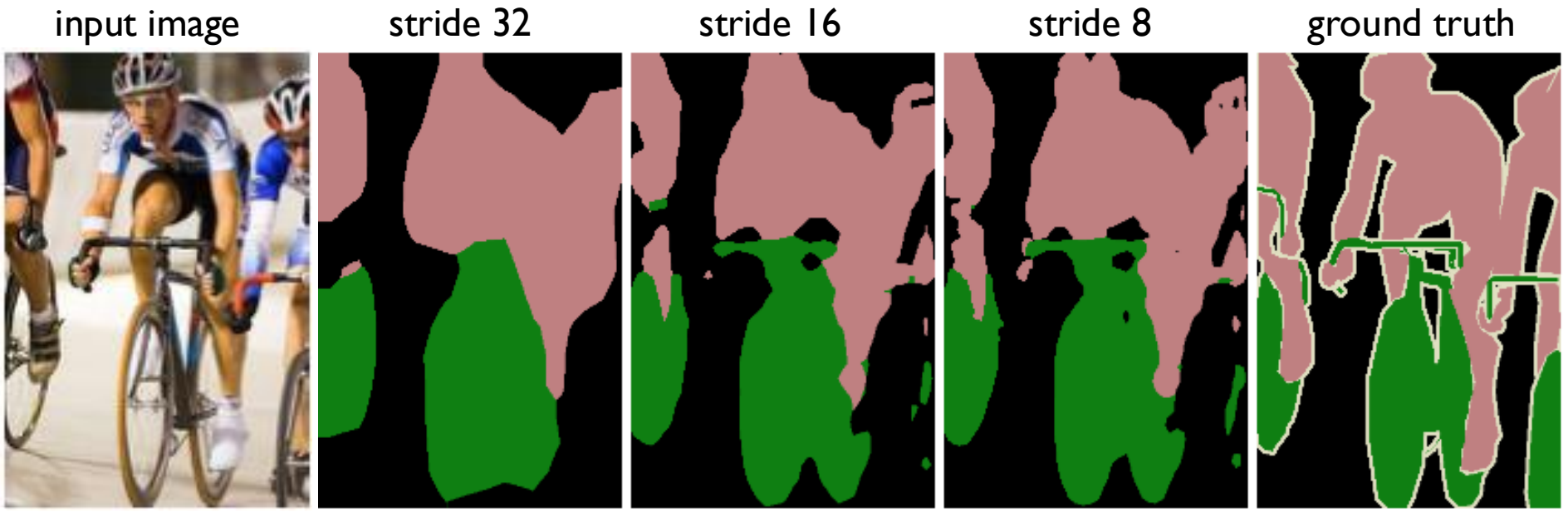


[Long et al.]

UNet [Ronneberger et al., 2015]



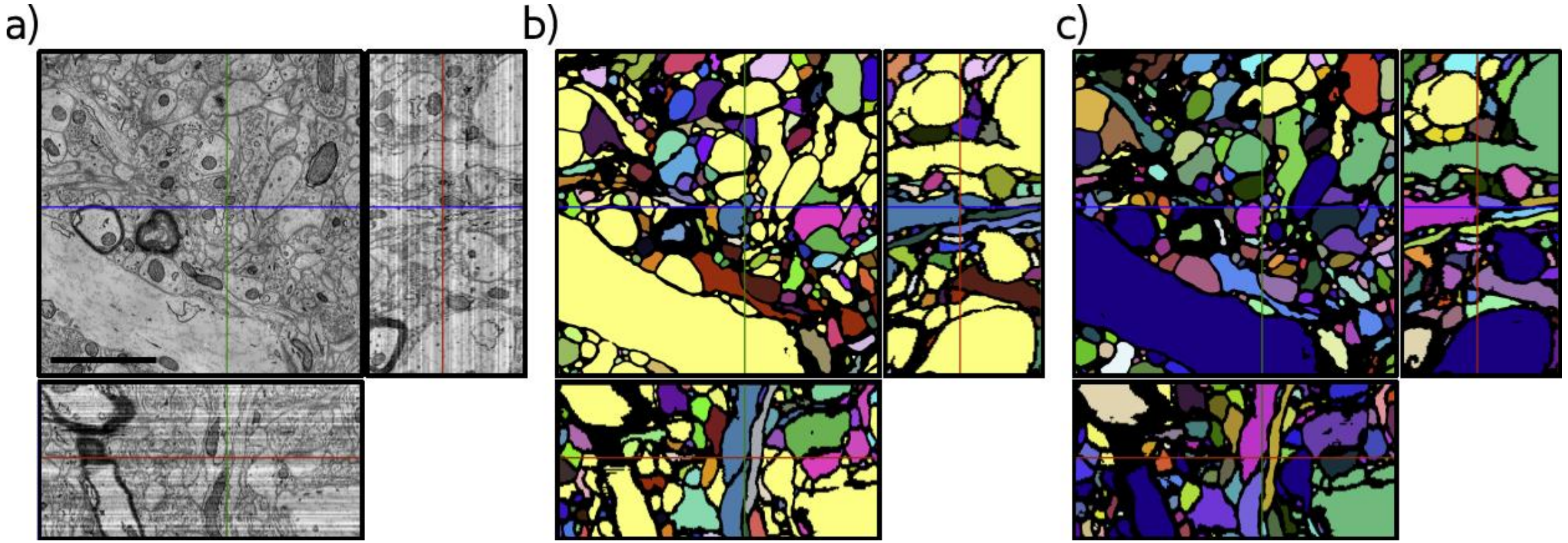
Skip layer refinement



U-Nets for everything!

Need a general encoder for images?

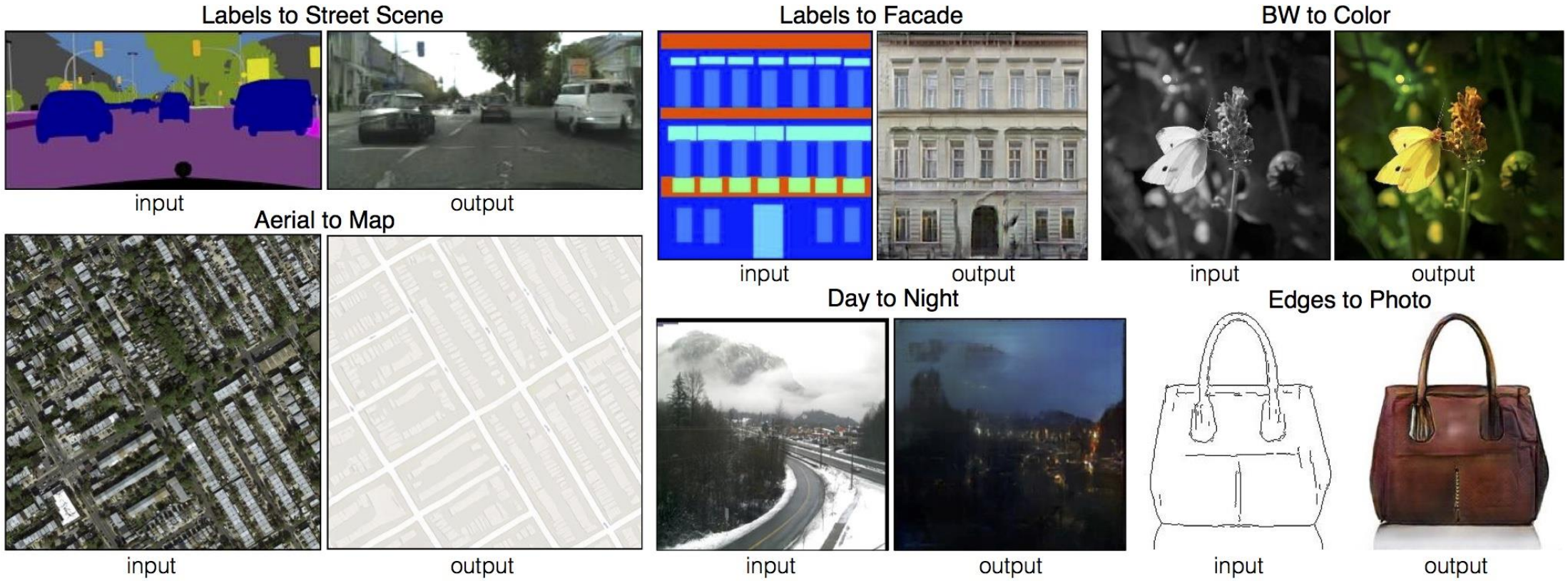
Need to map from one domain to another?



UNets for everything!

Need a general encoder for images?

Need to map from one domain to another?



[Isola Zhu, Zhou, Efros – Pix2Pix 2017] – a UNet with a GAN

UNets for everything!

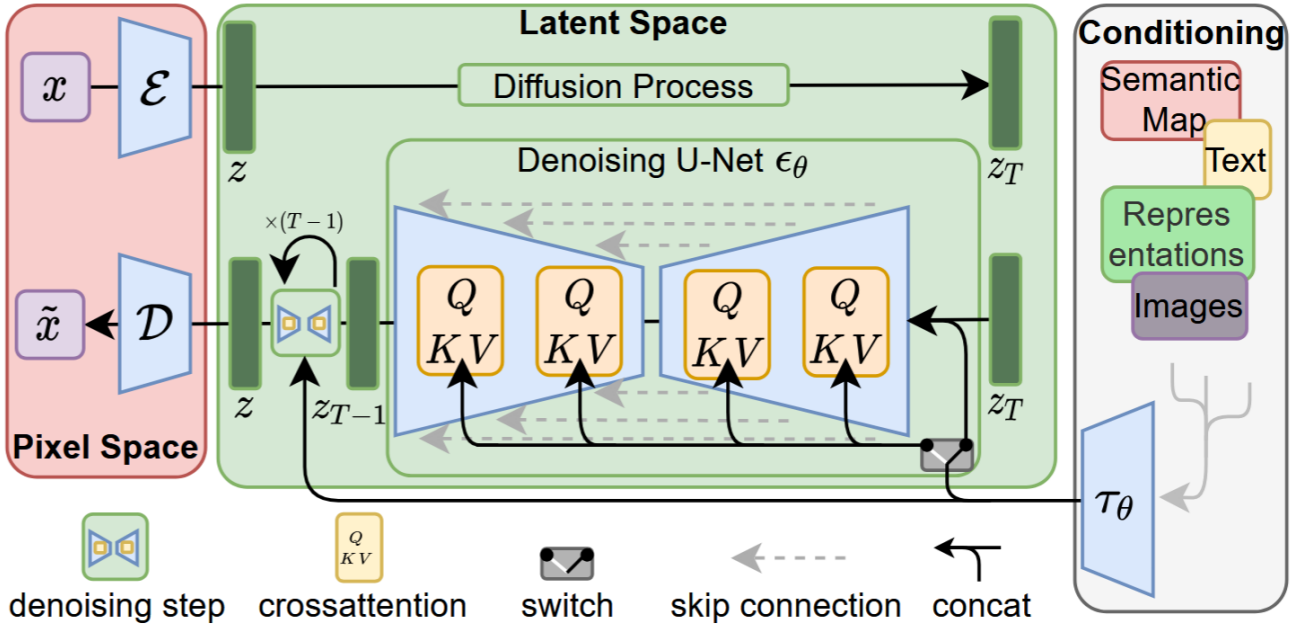
Need a general encoder for images?

Need a compressed image representation?



Stable Diffusion image generator

Has a UNet inside it as a general image latent space.

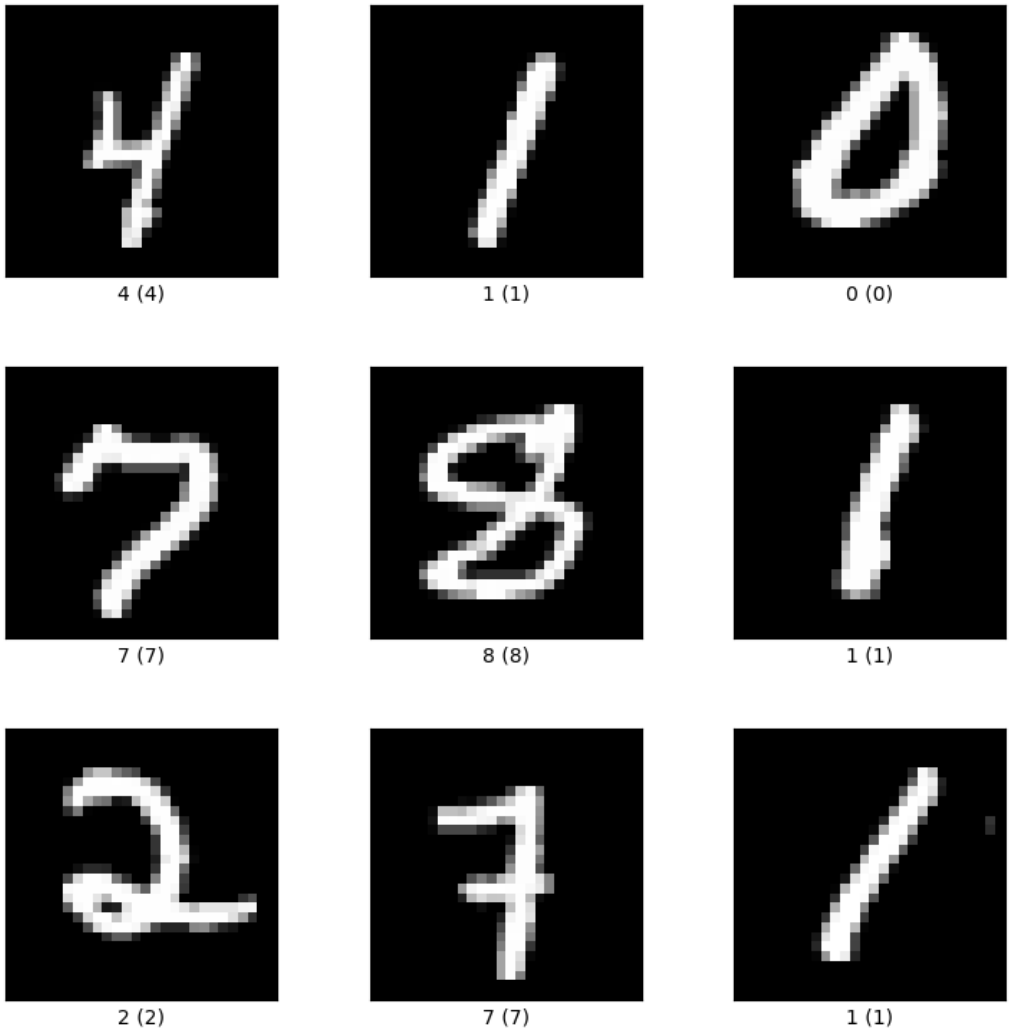


[Rombach, Blattmann, Lorenz, Esser, Ommer 2022]

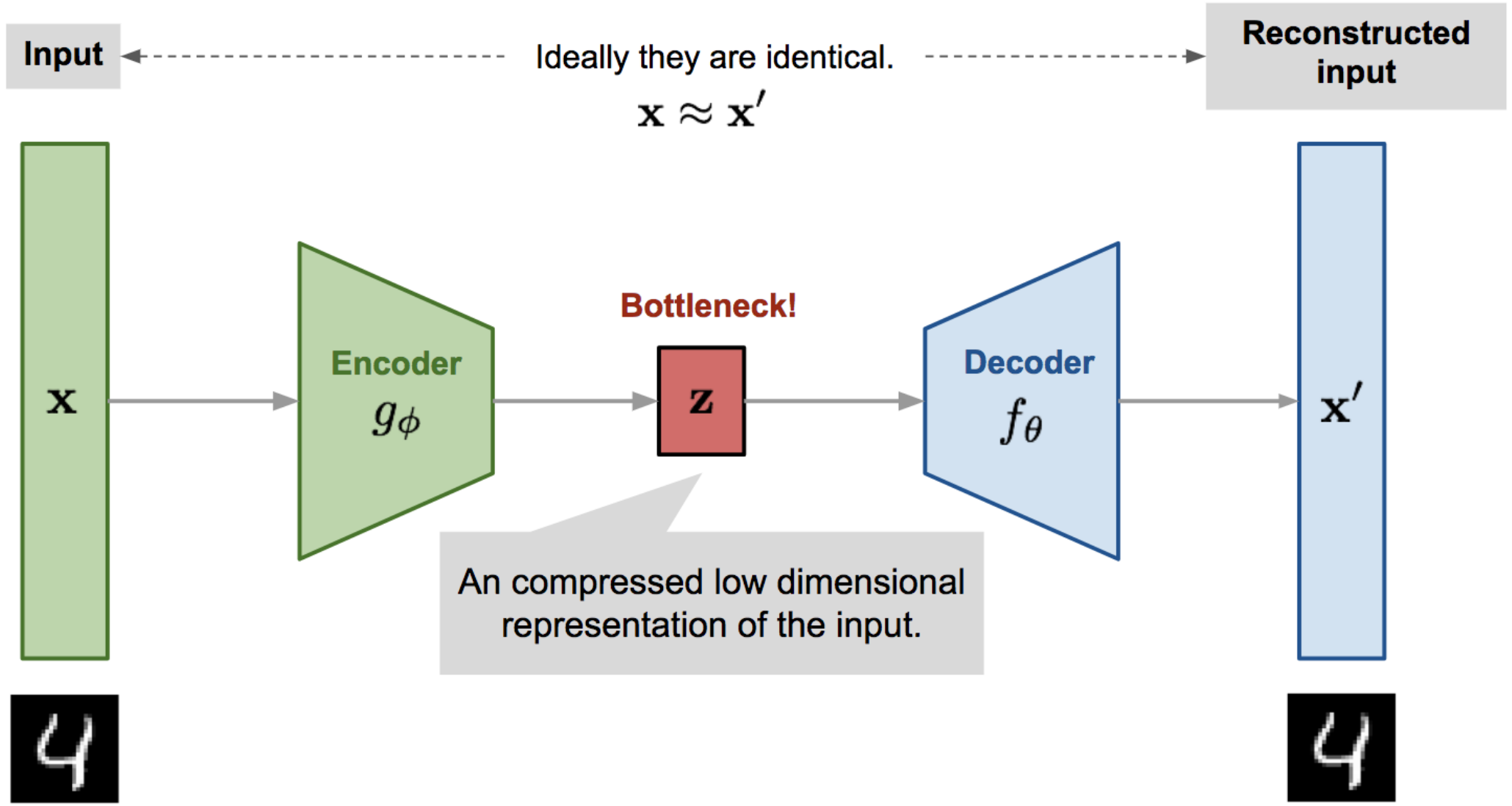
Auto-encoders

Unsupervsed / self-supervised with a reconstruction loss

MNIST



Autoencoders



2D Latent Space



Reconstruction



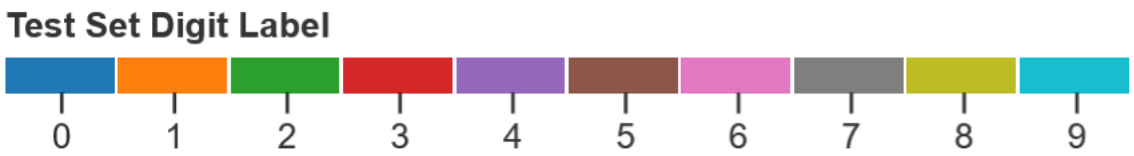
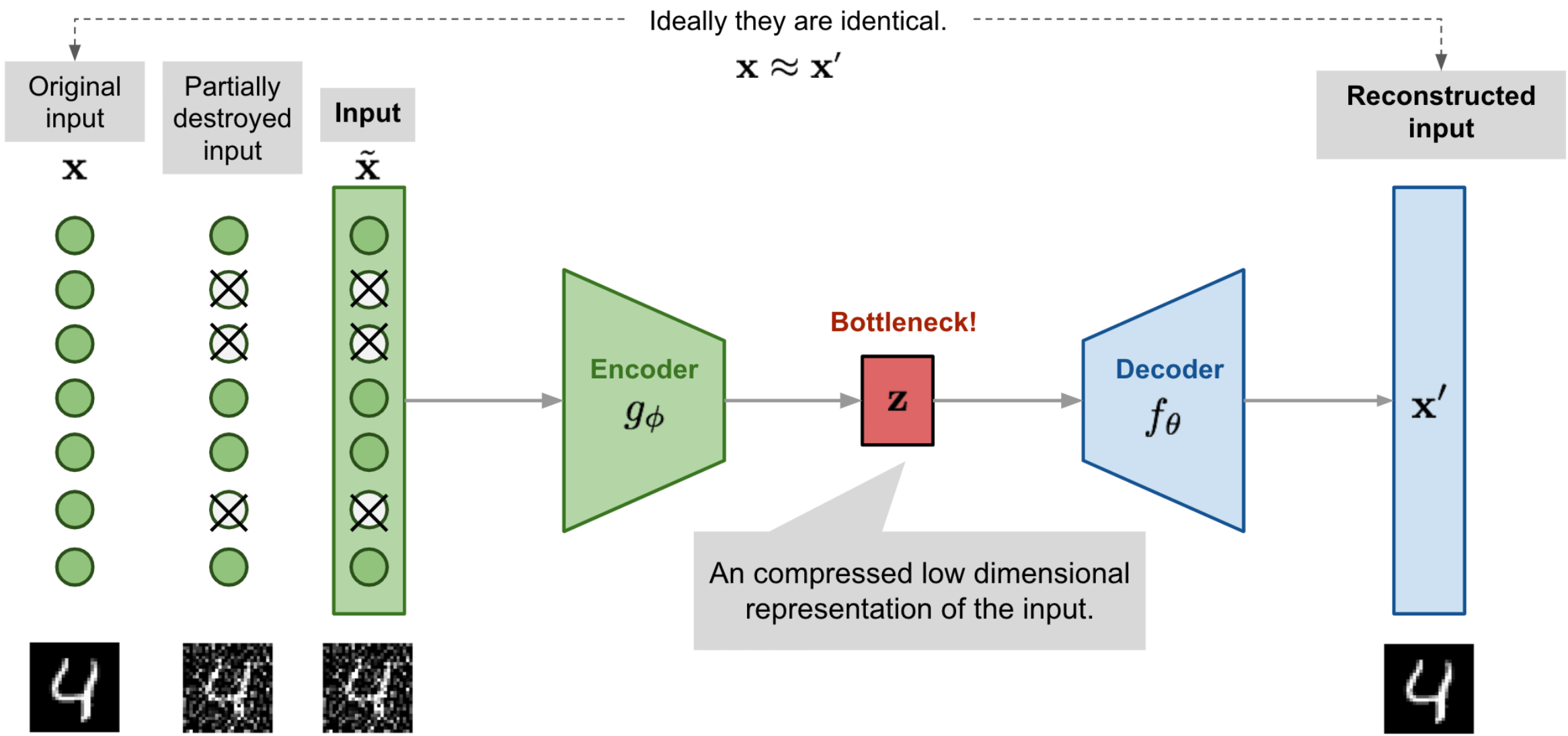


Figure 6: Hover over 2D Latent Space to see Reconstruction of a digit.

Denoising Autoencoders (Vincent et al. 2008)



What is 'in' an decoder?

Untrained Networks as Image Priors, Heckel & Hand, ICLR 2019

Deep Image Prior, Ulyanov et al. CVPR 2018

What is an image model?

Describes a class of signals it can represent.

Classical approach: DCT (discrete cosine transform) -> frequency space!

Represent natural images with fewer frequency coefficients than Nyquist limit.

Basis for JPEG compression!

Question: What kind of an image model is a CNN?

- …when trained on many images?
- …when trained on a single image?

The network structure itself is a 'prior'!

1. Pick a simple decoder network G with parameters C
2. Give it a fixed random input B_0 (never changes)
3. Optimize C so that G(C) matches the target image

The network structure constrains what images can be produced.

If the network has FEWER parameters than pixels ($N \ll n$):

→ It can represent natural images (smooth, structured)

→ It CANNOT represent random noise (unstructured)

→ *Built-in regularization, no early stopping needed!*



512 256 128 64 32 16 8



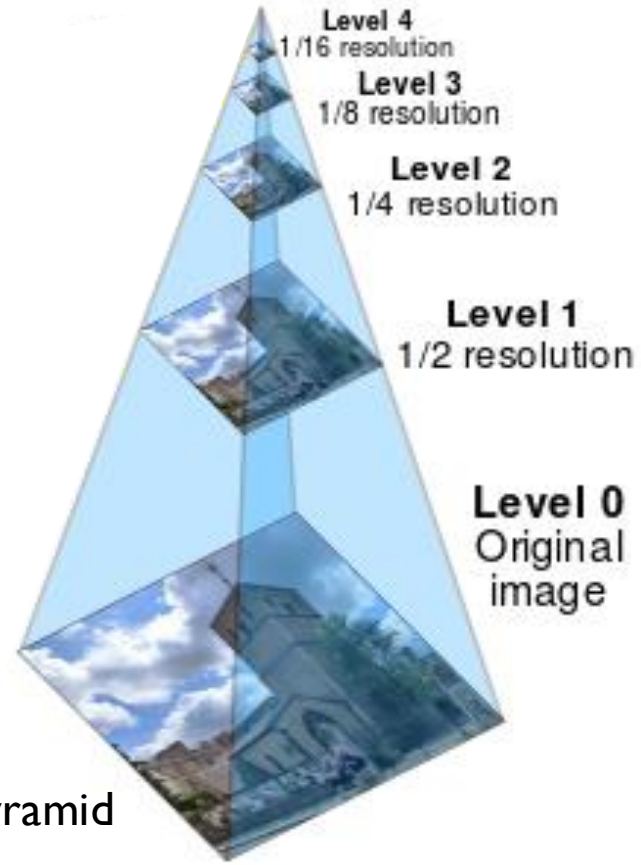


Image Pyramid

Peak Signal to Noise Ratio

If we had a noise free $m \times n$ image I and its noisy variant K:

Mean squared error $$MSE = \frac{1}{m\,n} \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} [I(i,j) - K(i,j)]^2$$

Work in log domain because scales of dynamic range are large.

$$\begin{aligned} PSNR &= 10 \cdot \log_{10} \left(\frac{MAX_I^2}{MSE} \right) \\ &= 20 \cdot \log_{10} \left(\frac{MAX_I}{\sqrt{MSE}} \right) \\ &= 20 \cdot \log_{10}(MAX_I) - 10 \cdot \log_{10}(MSE) \end{aligned}$$

Where MAX is peak of possible signal. E.G., 8bit, MAX =255.

[Wikipedia]

Peak Signal to Noise Ratio

Uncompressed image

JPEG compressed image



Q=90, PSNR 45.53dB

[Wikipedia]

Peak Signal to Noise Ratio

Uncompressed image

JPEG compressed image



Q=30, PSNR 36.81dB

[Wikipedia]

Peak Signal to Noise Ratio

Uncompressed image

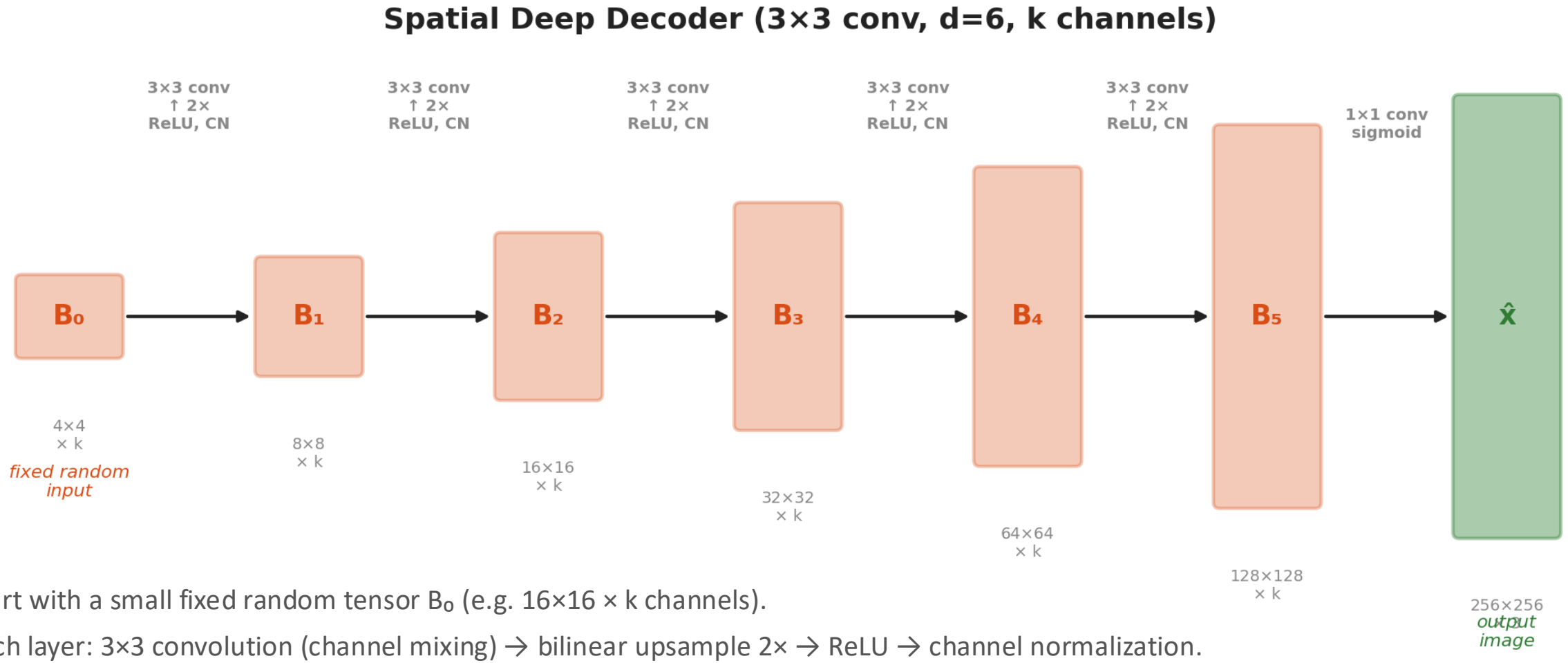
JPEG compressed image



Q=10, PSNR 31.45dB

[Wikipedia]

Example Decoder Architecture



Start with a small fixed random tensor B_0 (e.g. 16×16 × k channels).

Each layer: 3×3 convolution (channel mixing) → bilinear upsample 2× → ReLU → channel normalization.

Final layer: 1×1 convolution → sigmoid → output RGB image (512×512×3).

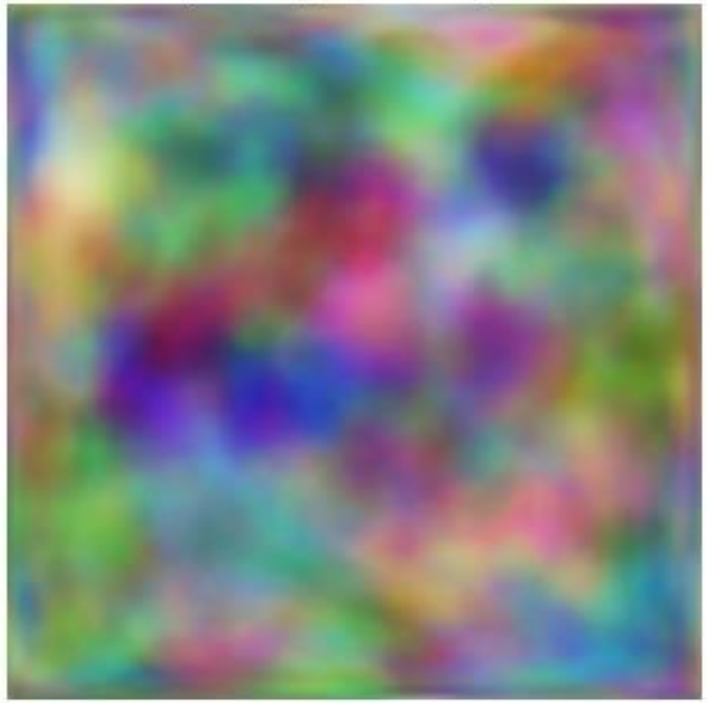
Noise is learned *last! (3x3 conv, bilinear, ReLU)*

Step 1/15000 | PSNR: 9.2 dB | lr: 0.00000

Noisy input



Spatial_3x3 (step 1)

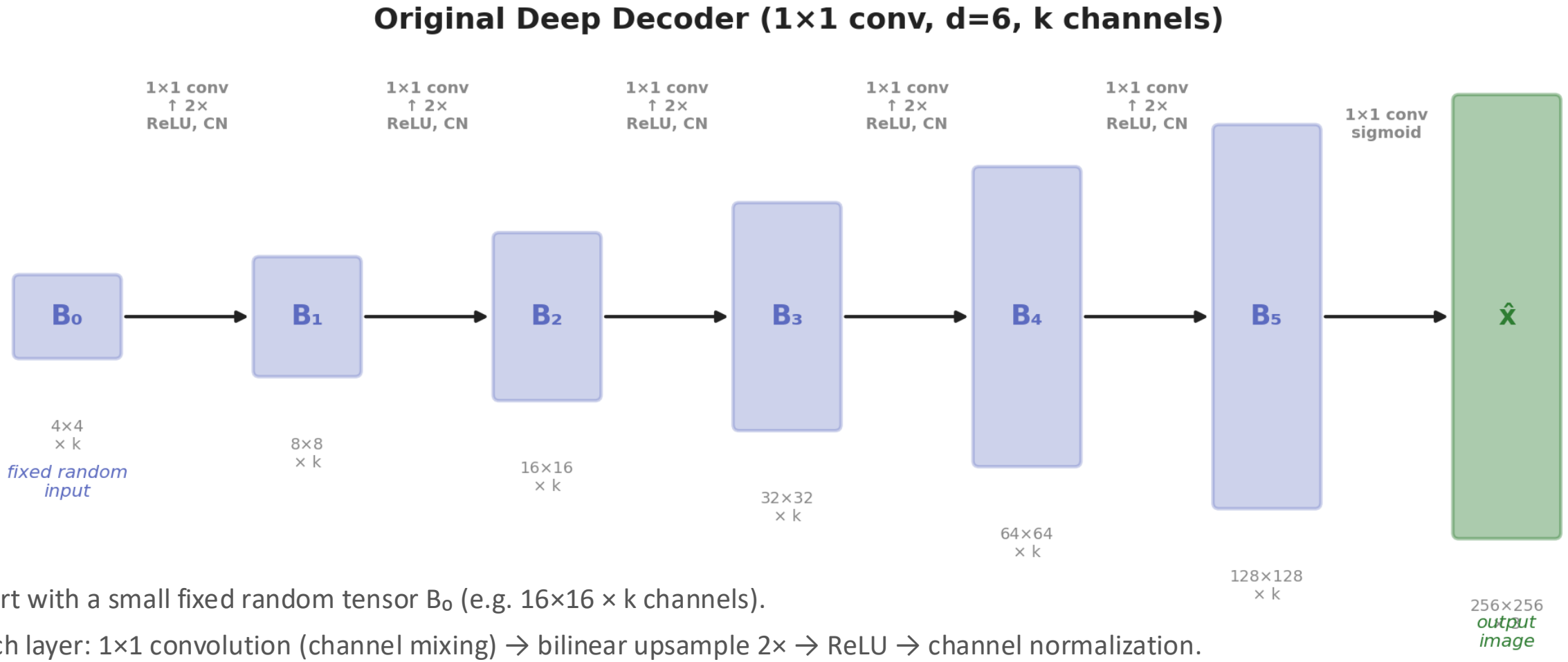


Clean target



Note: Uses cosine annealing learning rate decay.

Example Decoder Architecture



Start with a small fixed random tensor B_0 (e.g. 16×16 × k channels).

Each layer: 1×1 convolution (channel mixing) → bilinear upsample 2× → ReLU → channel normalization.

Final layer: 1×1 convolution → sigmoid → output RGB image (512×512×3).

No spatial convolutions! Only 1×1 (pixel-wise) + upsampling.

Noise is learned *last! (1x1 conv, bilinear, ReLU)*

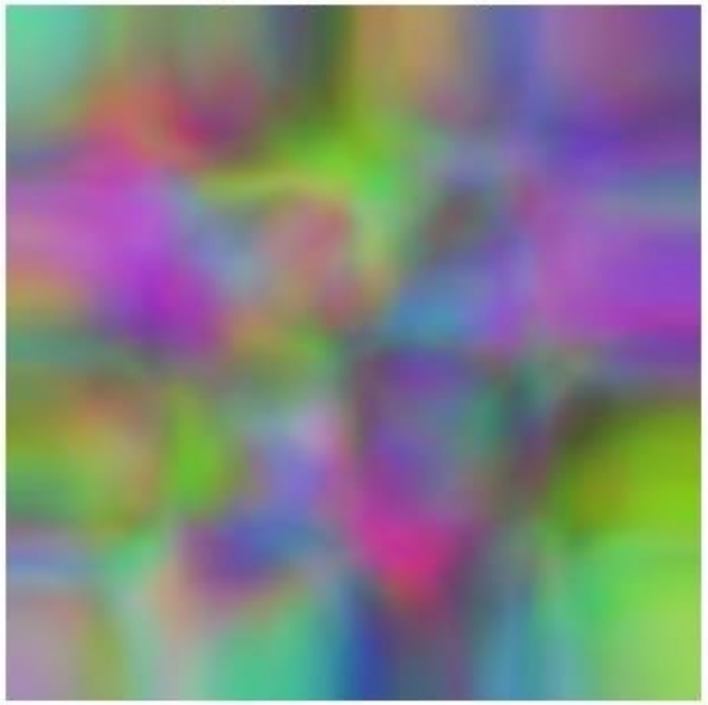
1 x 1 convolutions have fewer free parameters, more constrained.
Fits less noise than 3x3; achieves higher PSNR to clean target.

Step 1/15000 | PSNR: 9.4 dB | lr: 0.00000

Noisy input



Original_1x1 (step 1)



Clean target



Note: Uses cosine annealing learning rate decay.

Denoising

Denoising: σ = 25/255 | DD: 22.7 dB (k=128, 3000 iters)

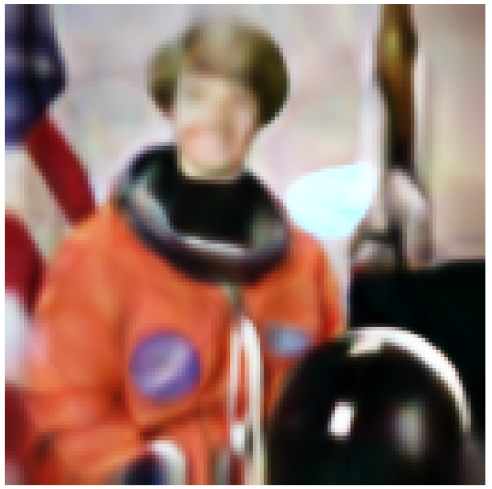
Original



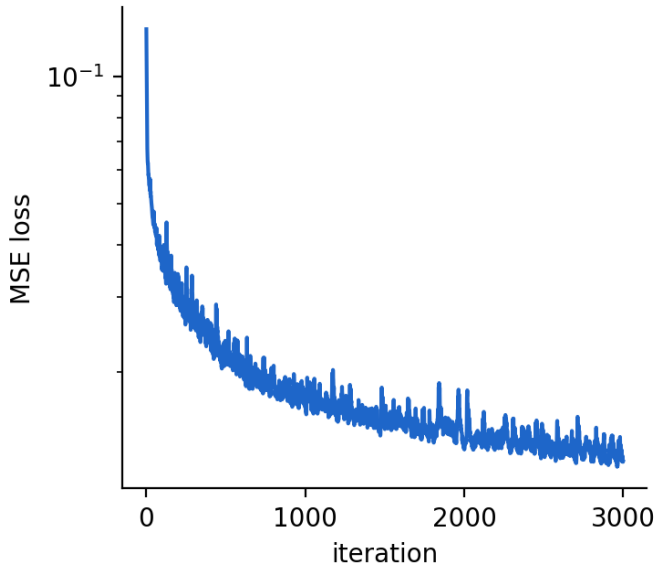
Noisy (20.8 dB)



Deep Decoder (22.7 dB)



Optimization



Noisy input: 20.8 dB → Deep Decoder output: 22.7 dB

No training data, no early stopping — just optimize and read off the result.

The underparameterized network physically cannot represent the noise.

Paper reports 29.6 dB on 512×512 astronaut (σ=25); we use 256×256 here for speed.

Also inpaints!

Inpainting: 50% pixels removed | DD: 22.0 dB (k=128, 3000 iters)

Original



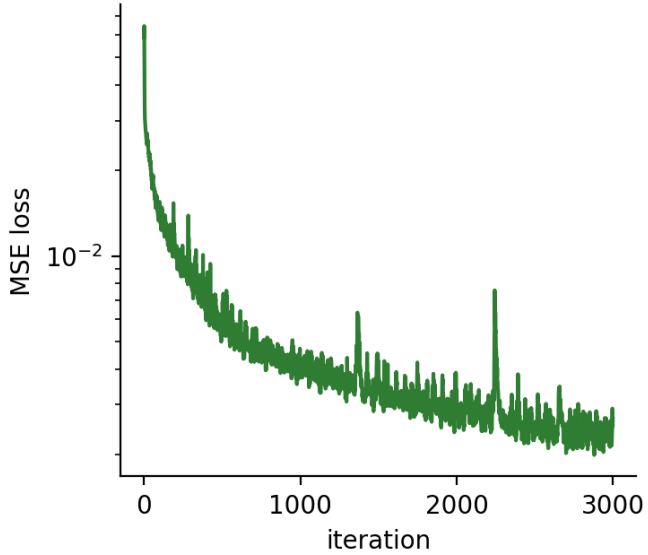
Masked (50% removed)



Deep Decoder (22.0 dB)



Optimization



50% of pixels removed → reconstruction: 22.0 dB

Same network, same code — only the loss changes: $L = || \text{mask} \cdot (G(C) - y) ||^2$

The network fills in missing pixels by leveraging its structural bias.

Key Takeaways

1. Neural networks can be image models WITHOUT any training.

The network structure itself encodes useful priors about natural images.

2. Upsampling = coarse-to-fine generation = image pyramid in reverse.

This is why the decoder naturally represents smooth, structured images.

3. Underparameterization provides automatic regularization.

Fewer parameters than pixels → can't fit noise → denoises for free.

4. Sometimes simpler is better.

Strip an autoencoder to just its decoder. Remove convolutions.

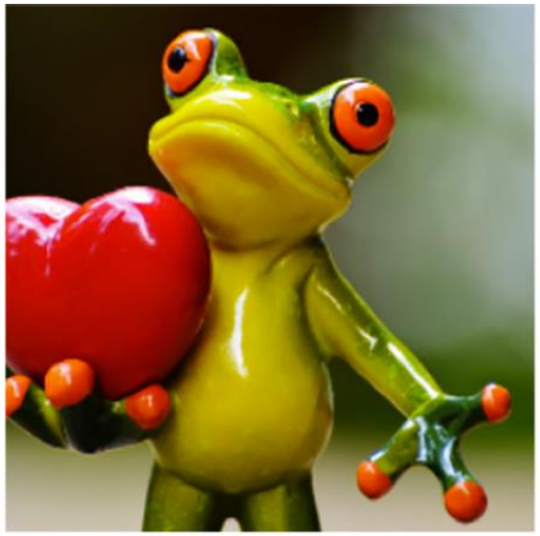
What's left still works — and is easier to understand and analyze.

Other model inspection approaches

Additional slides

Is there a model agnostic way?

Local Interpretable
Model-agnostic Explanations (LIME)



Original Image

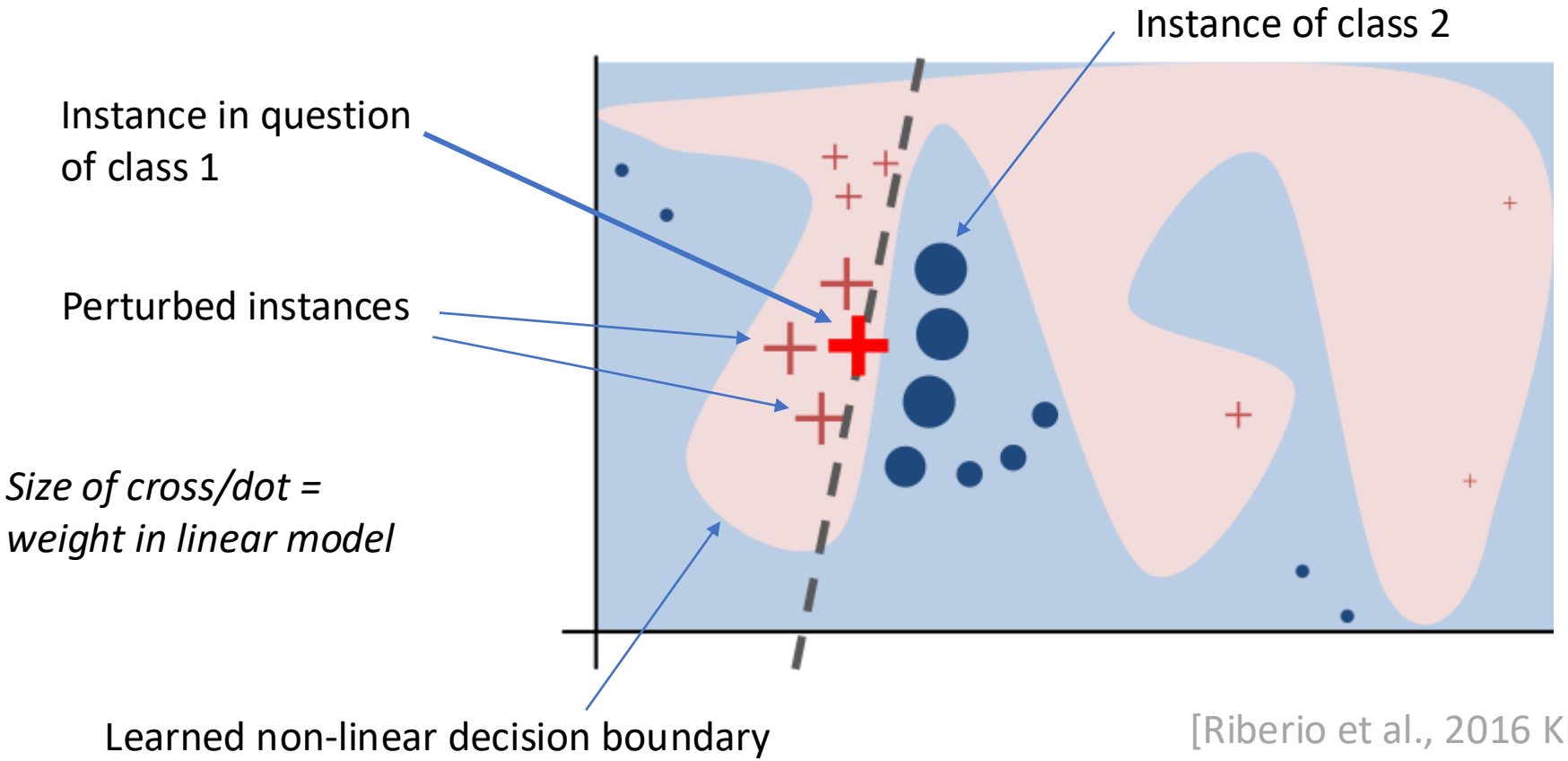
Which image regions were important for classifying this as "tree frog"?

Which for "pool table"?

[Riberio et al., 2016 KDD]

What is LIME doing?

Explaining complex decision boundary of black box model by a linear model trained around local perturbations of an instance.



[Riberio et al., 2016 KDD]

Local Interpretable Model-agnostic Explanations (LIME)

Step 1: Split the image up into superpixels



Original Image



Interpretable Components

[Riberio et al., 2016 KDD]

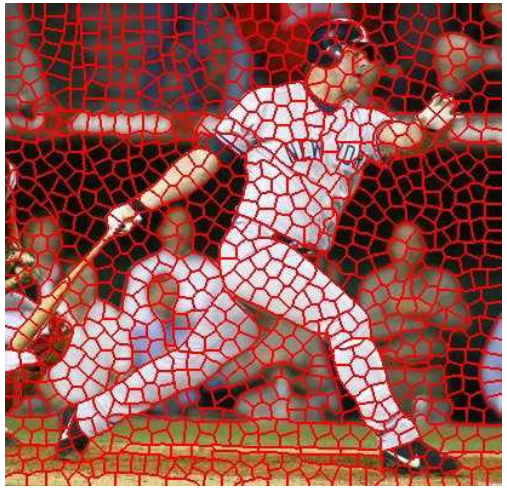
Superpixels: Mid-level clustering





[Felzenszwalb and Huttenlocher 2004]





Superpixels!

[Shi and Malik 2001]

[Hoiem et al. 2005, Mori 2005]

Superpixel algorithm (SLIC)

Approach: k-means clustering

Setup:

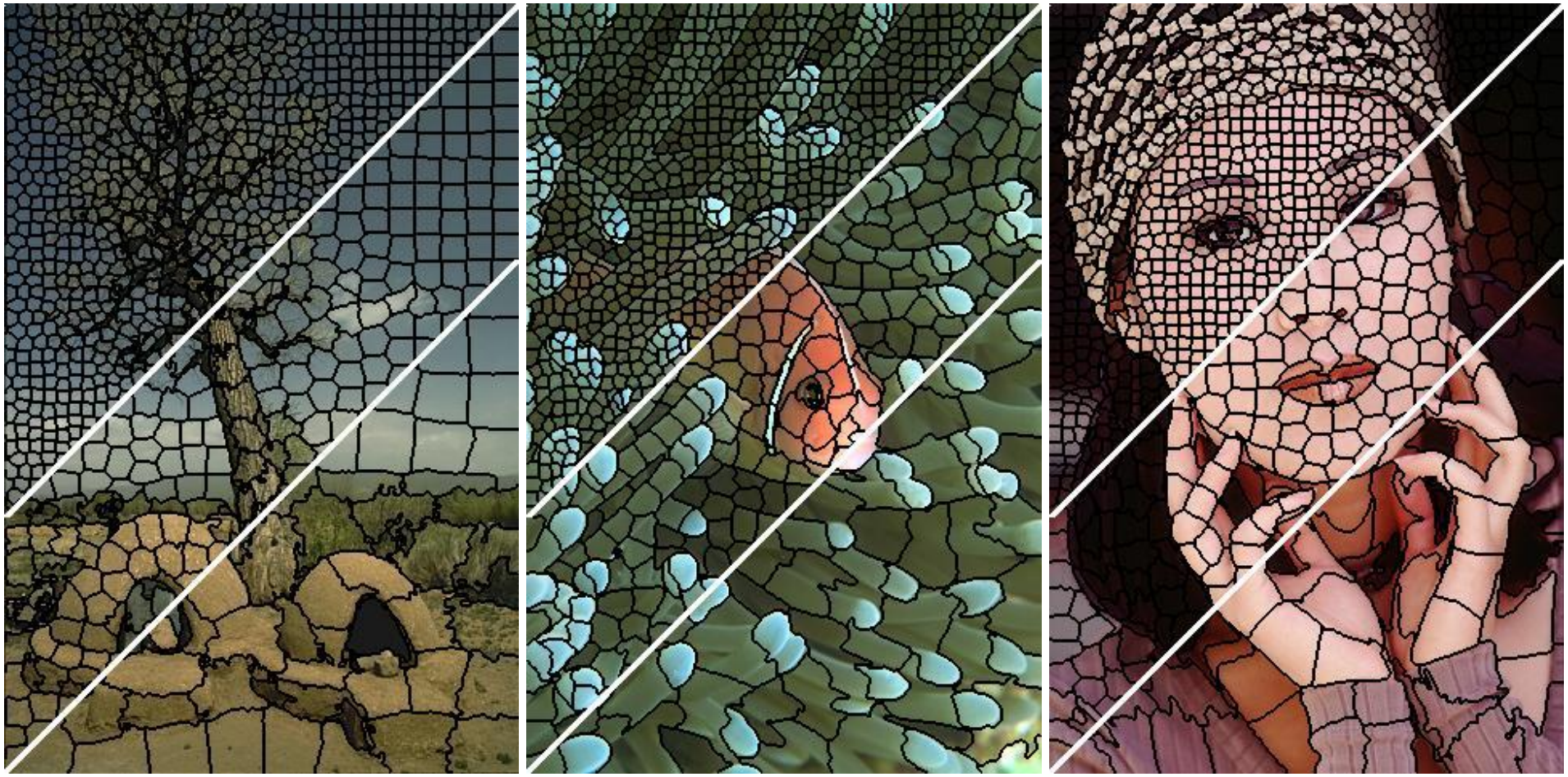
1. Features: l,a,b (color) and x,y (position)
2. Metric: Euclidean distance with spatial normalization for x,y

Algorithm:

1. Distribute 'seed' points (initial k-clusters) across image in a regular grid
2. Run k-means until convergence
3. Split clusters that are disconnected spatially
4. Assign superpixels by clusters

[Simplified version of 'SLIC' superpixels – Achanta et al. 2010]

SLIC Superpixel Results (varying k)



Local Interpretable Model-agnostic Explanations (LIME)

Step 1: Split the image up into superpixels



Original Image



Interpretable Components

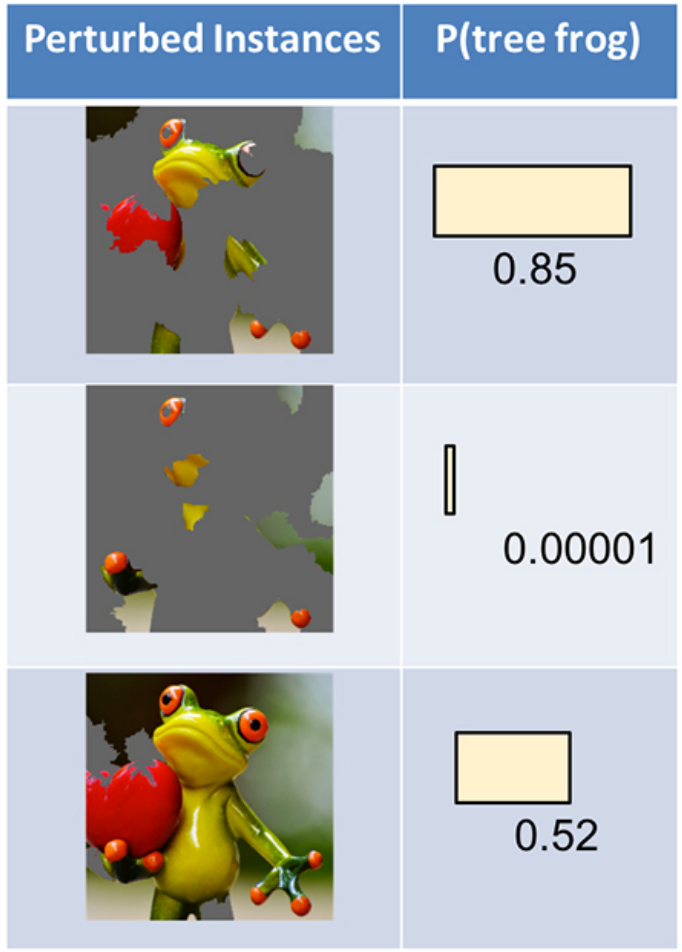
[Riberio et al., 2016 KDD]

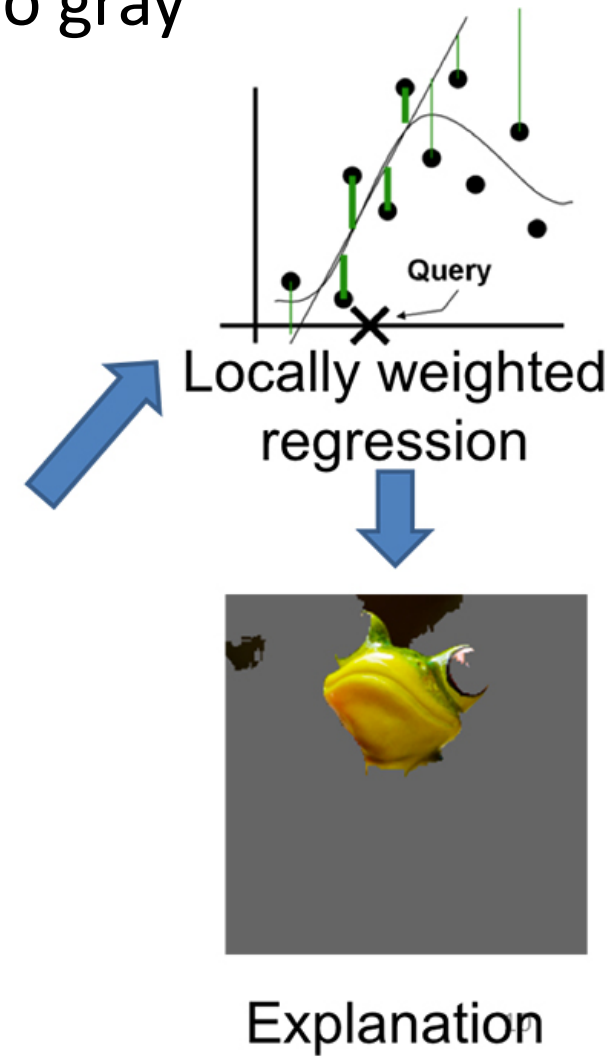
Local Interpretable Model-agnostic Explanations (LIME)

Step 2: Perturb instances – randomly set superpixels to gray and generate new dataset



Original Image
P(tree frog) = 0.54



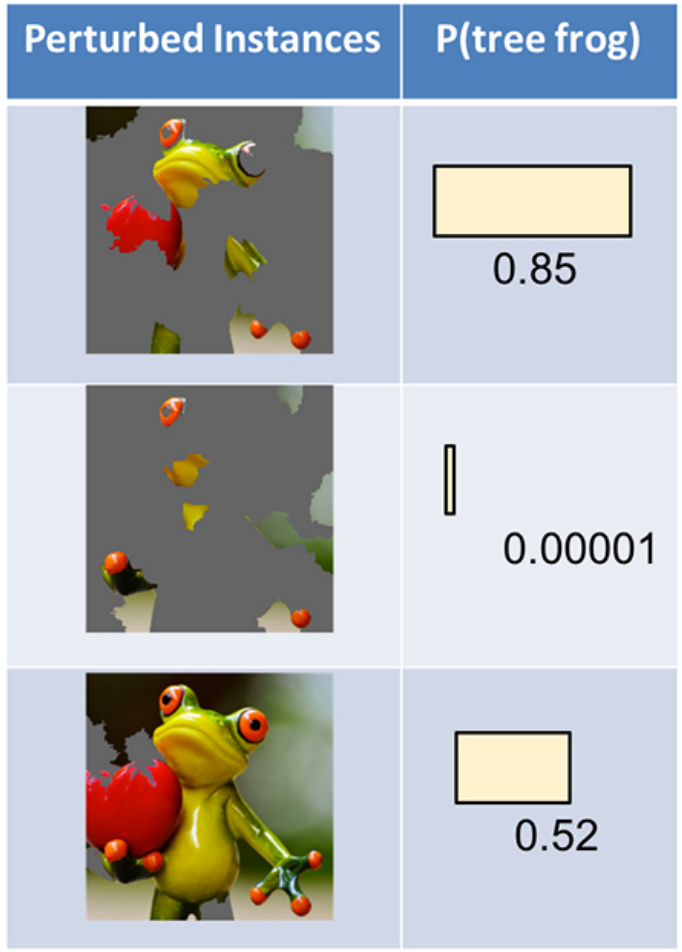


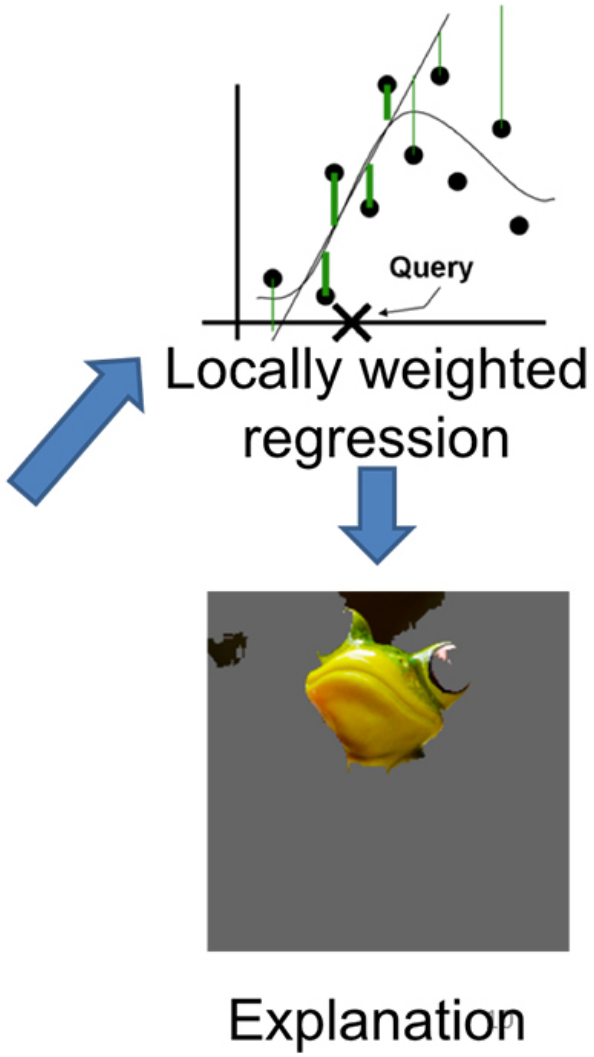
Local Interpretable Model-agnostic Explanations (LIME)

Step 3: Classify the perturbed instances



Original Image
P(tree frog) = 0.54



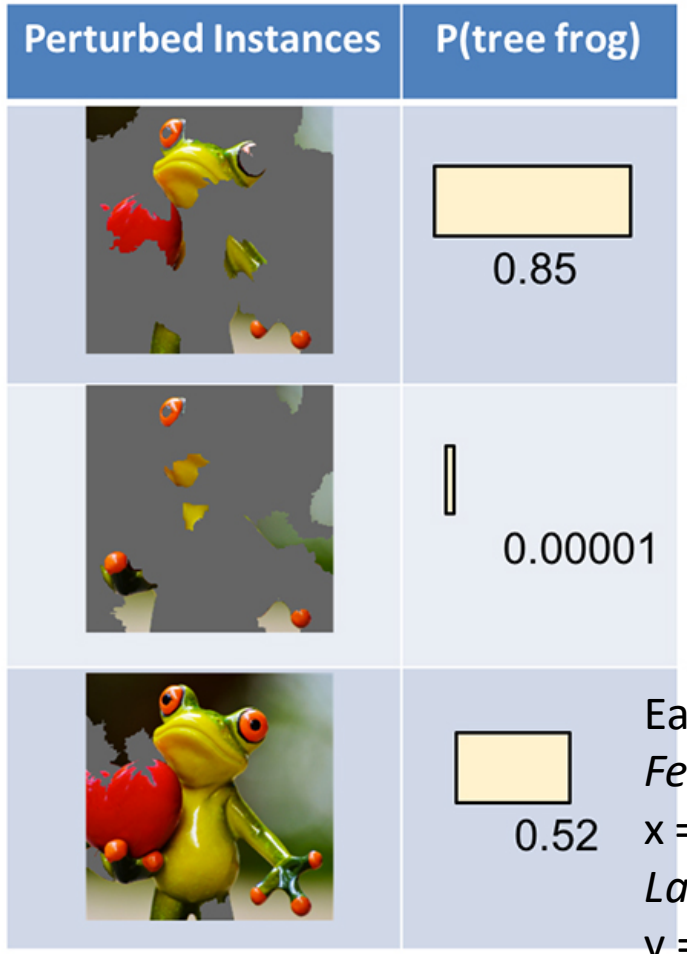


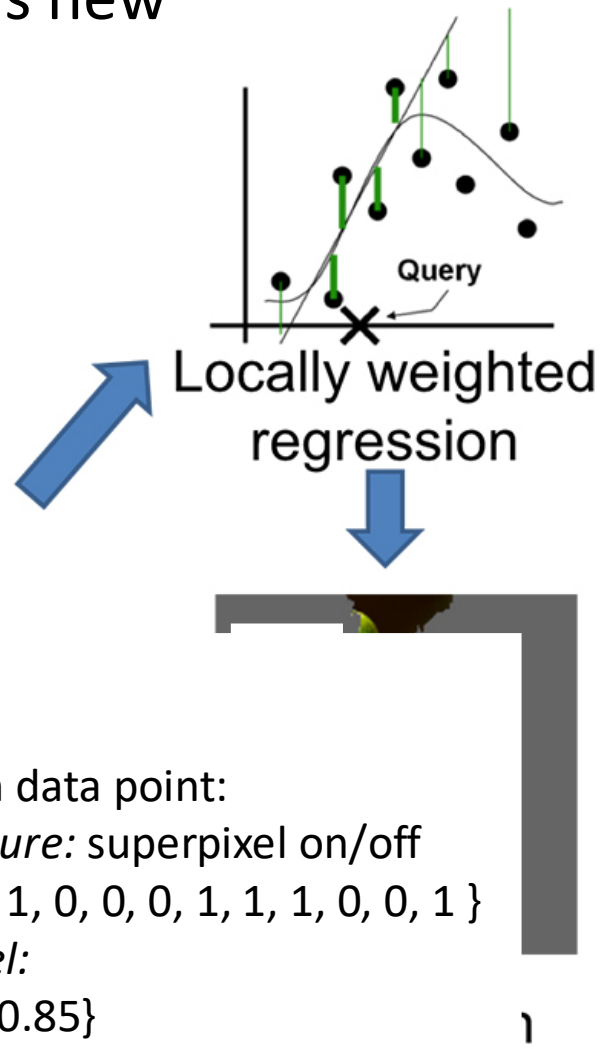
Local Interpretable Model-agnostic Explanations (LIME)

Step 4: Learn a linear classification model (K-LASSO) on this new dataset that weights outputs.



Original Image
P(tree frog) = 0.54

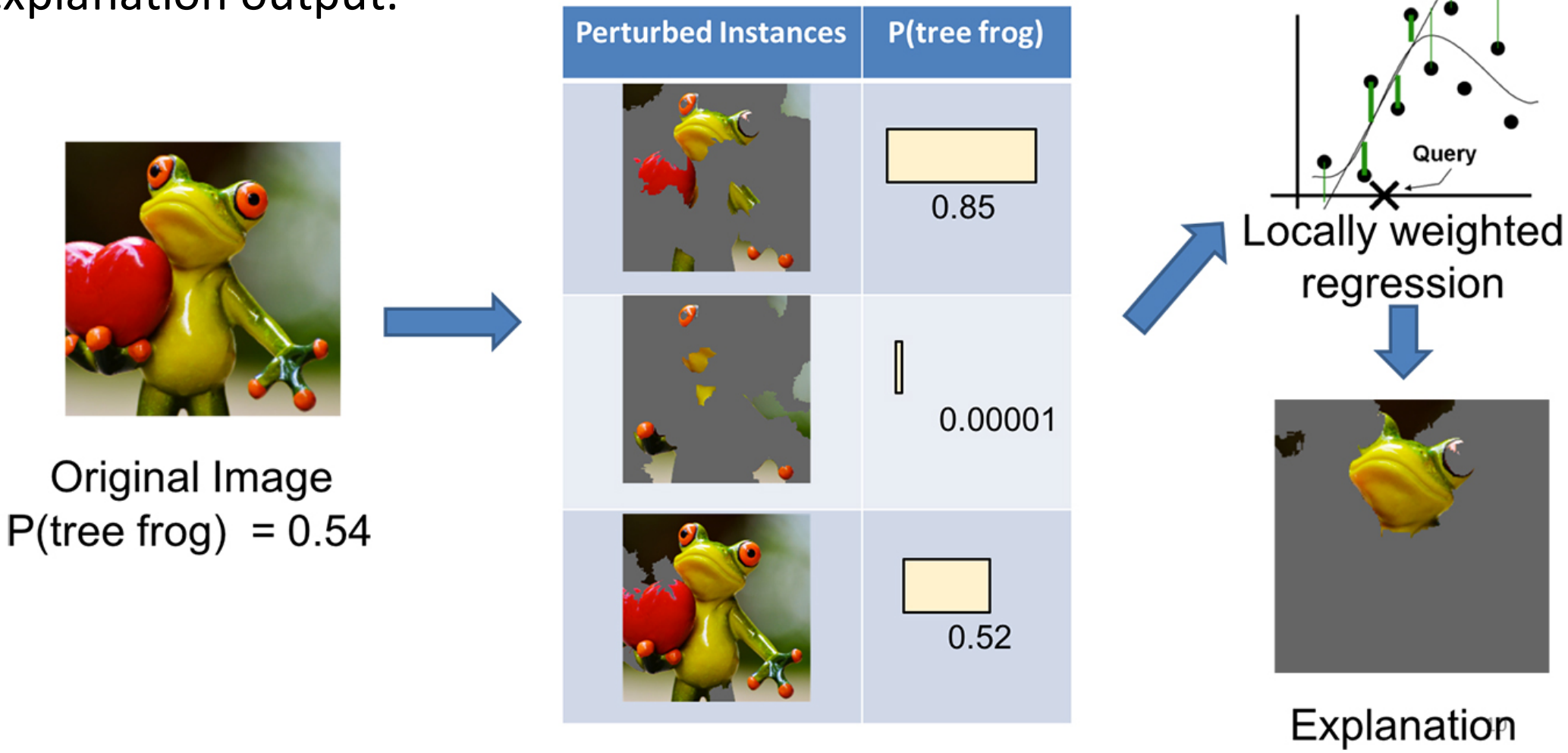




Each data point:
Feature: superpixel on/off
x = { 1, 0, 0, 0, 1, 1, 1, 0, 0, 1 }
Label:
y = {0.85}

Local Interpretable Model-agnostic Explanations (LIME)

Step 5: Keep only intersection of high-weight examples for final explanation output.



LIME: Which areas caused the classifications?



Most confident: “Tree frog”

P() = 0.54



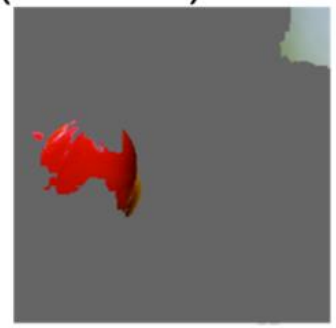
2nd confident: “Pool table”

P() = 0.07



3rd confident: “Balloon”

P() = 0.05



Why useful?

Linear model on superpixels might be easier to interpret.

Perturbation + local approximation trick works for *any classifier* – *classifier is treated as a black box*.

Output explanations do not require underlying network representation to be interpretable itself.

Explanations do not require any knowledge of the inner workings of the network – only the output classifications
Operates on true black boxes

[Riberio et al., 2016 KDD]